

UNION OF EDUCATION

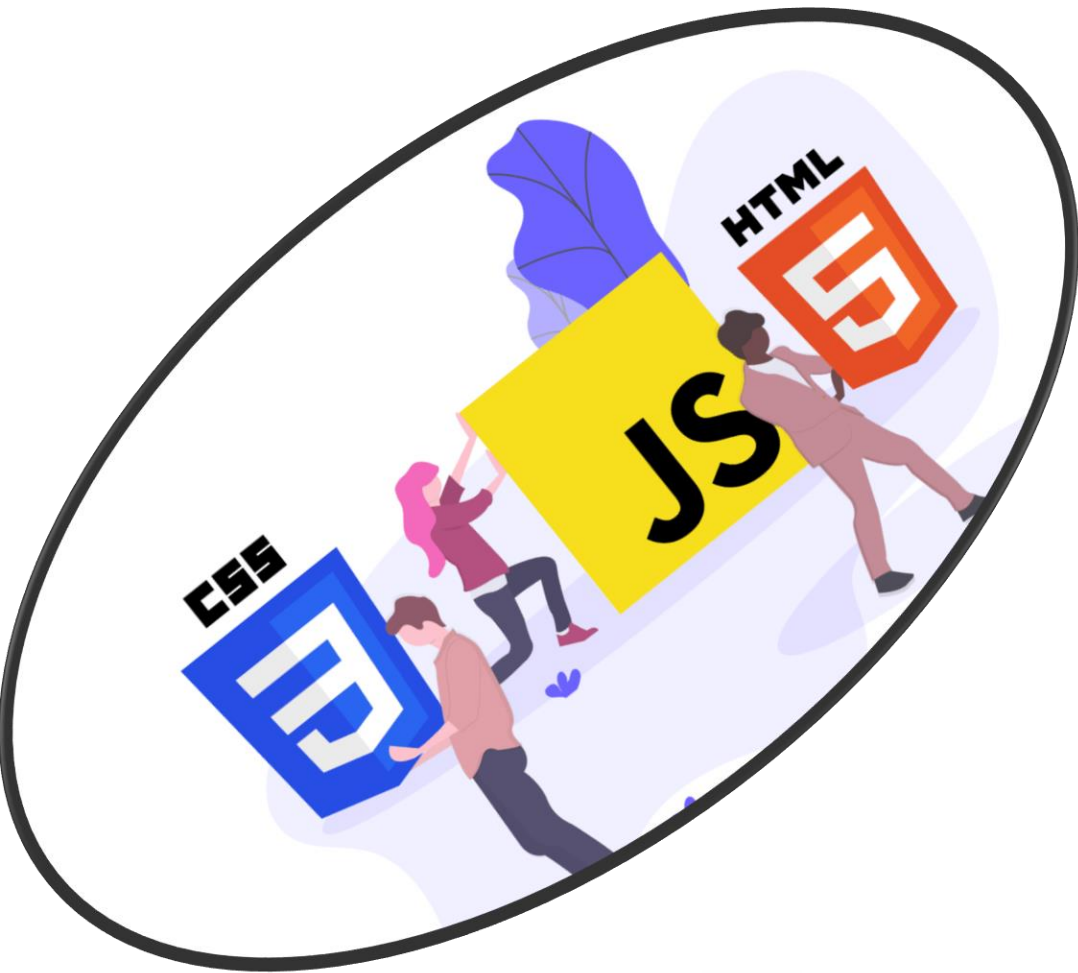
FORMATION

HTML CSS

JAVASCRIPT

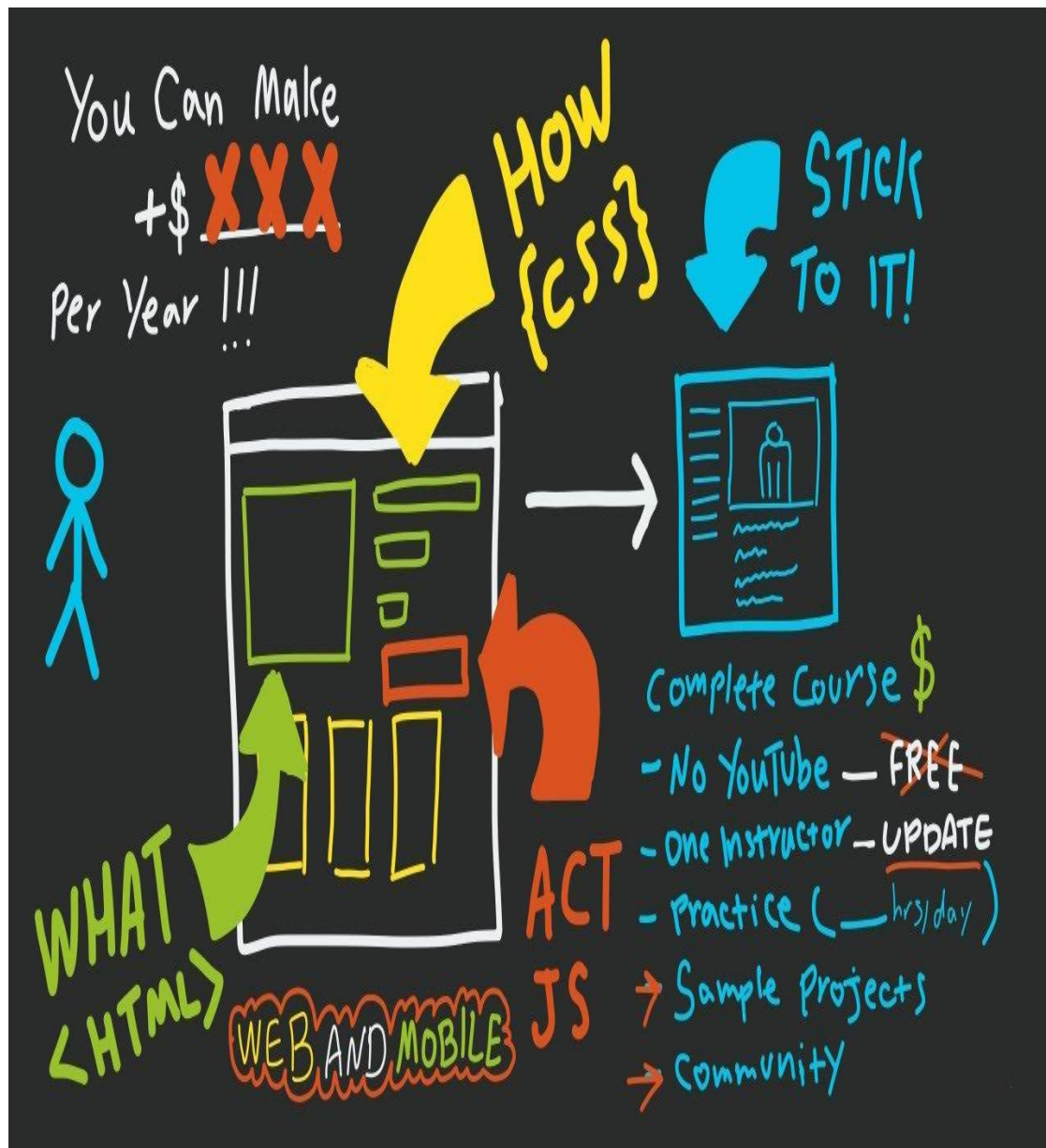


OBJECTIF



À la fin de la formation, vous serez capable de :

- Utiliser du code HTML ;
- Structurer une page web en HTML ;
- Utiliser du code CSS ;
- Mettre en forme une page web en CSS ;
- Organiser les éléments d'une page web grâce au CSS ;
- Modifier l'agencement d'une page HTML avec CSS ;
- Adapter une page pour les petites résolutions en CSS.
- Mémoriser les bases de JavaScript et de son utilisation pour le DOM
- Gérer les évènements et les manipulations dynamiques



PROGRAMME

INTRODUCTION

PARTIE 1 - LES BASES DE HTML5

PARTIE 2 - METTEZ EN FORME VOS PAGES AVEC CSS

PARTIE 3 - MISE EN PAGE AVEC HTML ET CSS

PARTIE 4 - DÉCOUVREZ DES FONCTIONNALITÉS ÉVOLUÉES

PARTIE 5 - JAVASCRIPT ET MANIPULATION DU DOM

LE FONCTIONNEMENT DES SITES WEB

Avec le navigateur, vous pouvez consulter n'importe quel site web. Voici par exemple un navigateur affichant le célèbre site web Wikipédia :



Les langages HTML et CSS sont à la base du fonctionnement de tous les sites web. Quand vous consultez un site avec votre navigateur, il faut savoir que, en coulisses, des rouages s'activent pour permettre au site web de s'afficher. L'ordinateur se base sur ce qu'on lui a expliqué en HTML et CSS pour savoir ce qu'il doit afficher, comme le montre la figure suivante.

Langages
HTML et CSS



Traduction par
l'ordinateur



Résultat visible
à l'écran



LES RÔLES DE HTML ET CSS

- **HTML** (*HyperText Markup Language*) : il a fait son apparition dès 1991 lors du lancement du Web. Son rôle est de gérer et organiser le contenu. C'est donc en HTML que vous écrirez ce qui doit être affiché sur la page : du texte, des liens, des images... Vous direz par exemple : « Ceci est mon titre, ceci est mon menu, voici le texte principal de la page, voici une image à afficher, etc. » ;
- **CSS** (*Cascading Style Sheets*, aussi appelées *feuilles de style*) : le rôle du CSS est de gérer l'apparence de la page web (agencement, positionnement, décoration, couleurs, taille du texte...). Ce langage est venu compléter le HTML en 1996.

- Vous pouvez très bien créer un site web uniquement en HTML, mais celui-ci ne sera pas très beau : l'information apparaîtra « brute ». C'est pour cela que le langage CSS vient toujours le compléter.

HTML
(pas de CSS)



HTML + CSS

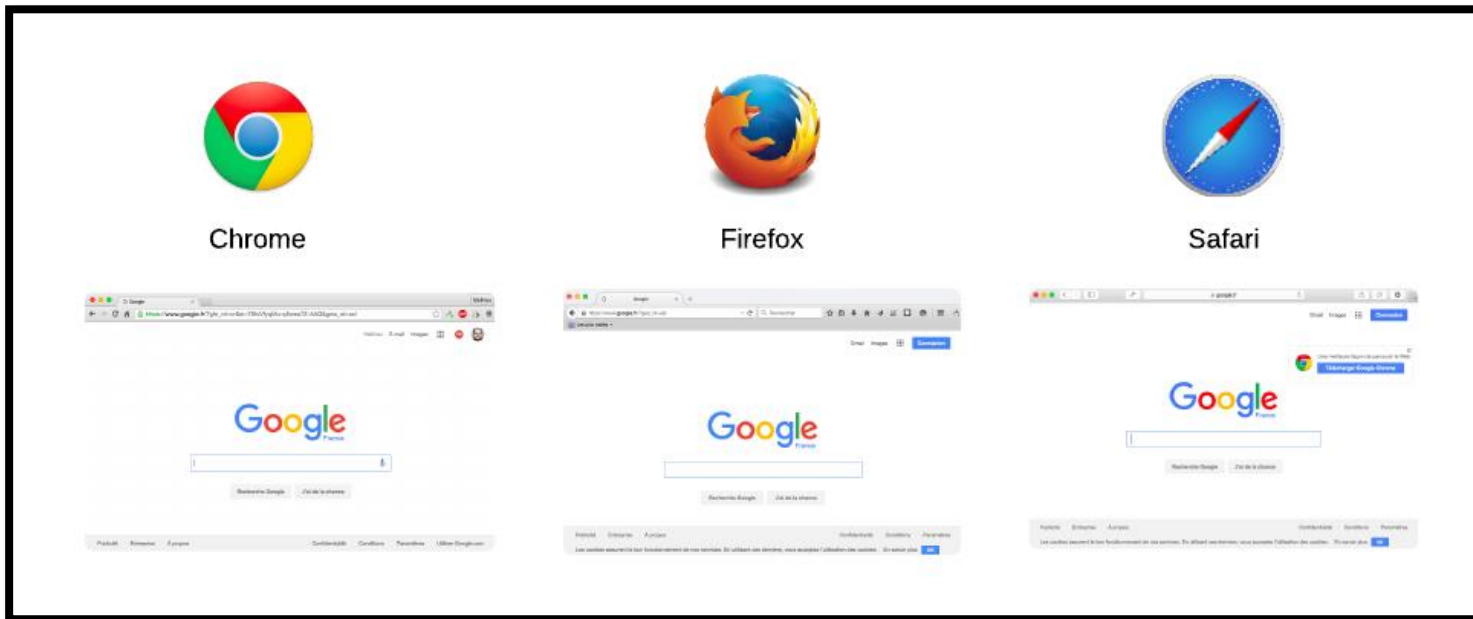


L'ÉDITEUR DE TEXTE

- Il existe effectivement de nombreux logiciels dédiés à la création de sites web
- **VS code Text : mon éditeur**
- **VS code** est un éditeur de texte devenu très populaire parmi les développeurs. On l'utilise aussi bien pour développer en HTML et CSS que dans d'autres langages (Python, Ruby, etc.). Il fonctionne sur Windows, Mac OS X et Linux.
- Il a l'avantage d'être simple, épuré et facile à lire dès le départ. Pas de centaines de boutons dont on ne comprend pas à quoi ils servent.

Les navigateurs

- Le navigateur est le programme qui nous permet de voir les sites web
- Le travail du navigateur est de lire le code HTML et CSS pour afficher un résultat visuel à l'écran. Si votre code CSS dit « Les titres sont en rouge », alors le navigateur affichera les titres en rouge. Le rôle du navigateur est donc essentiel !

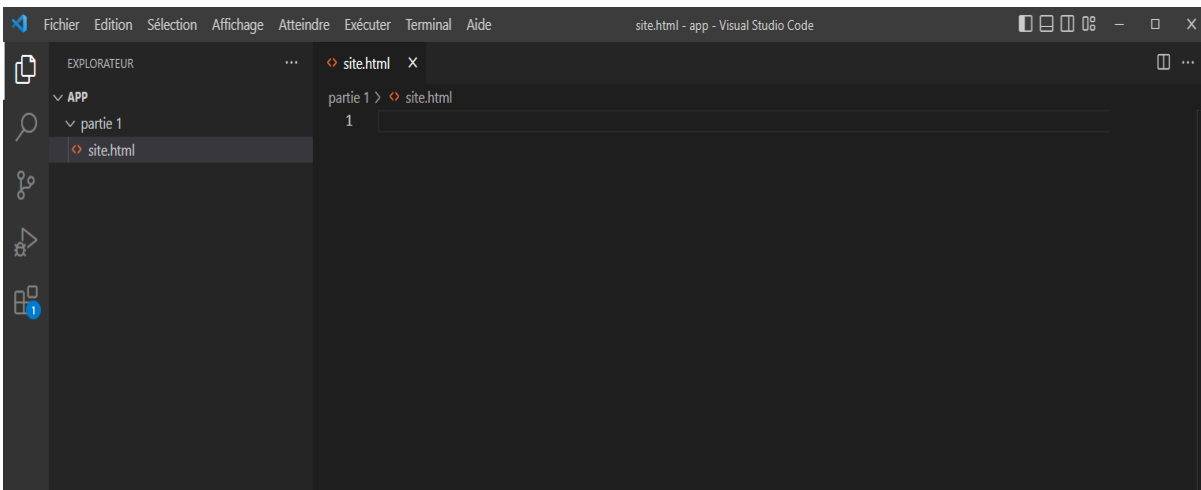


PARTIE 1 - LES BASES DE HTML5

1. Organisez votre texte
2. Créez des liens
3. Insérez des images

CRÉEZ VOTRE PREMIÈRE PAGE WEB EN HTML

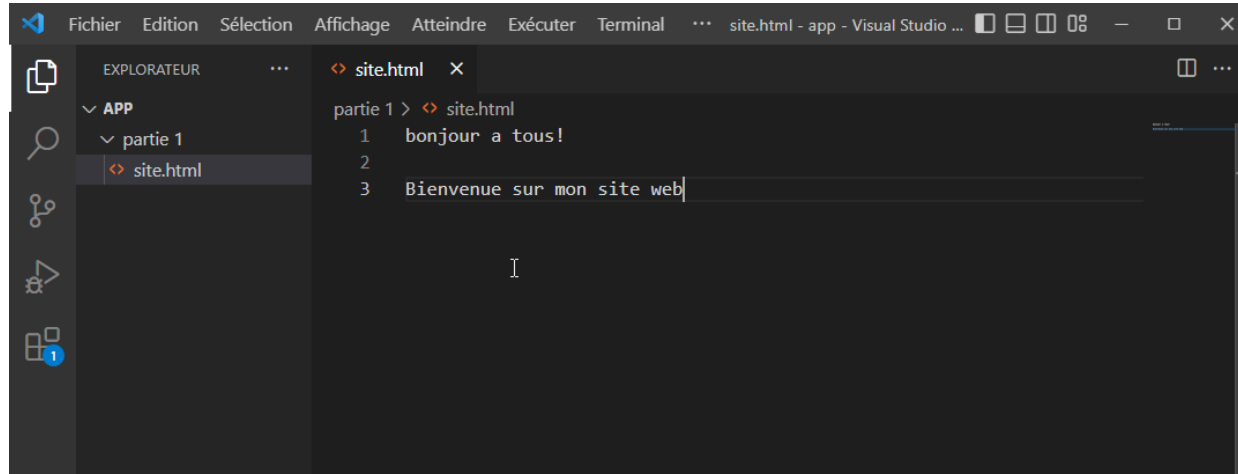
nous allons créer notre site dans un éditeur de texte , VS CODE



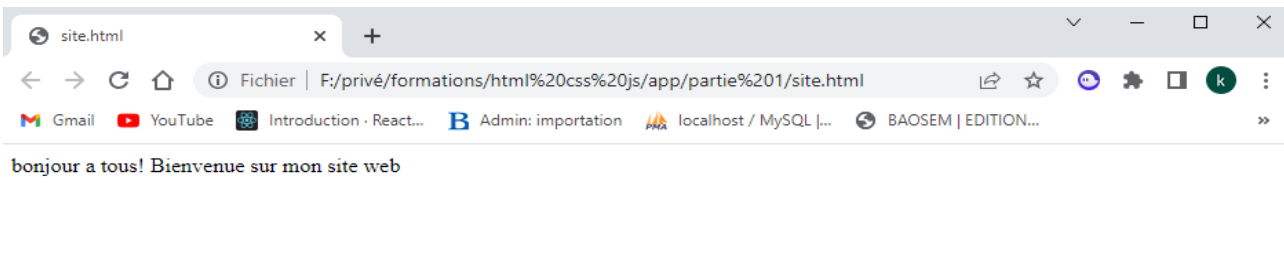
Pour créer une page web il faut créer un fichier avec l'extention html dans notre cas notre fichier est site.html

Les extensions indiquent à votre ordinateur quelle application a créé ou peut ouvrir le fichier et quelle icône utiliser pour le fichier. Par exemple, l'extension *docx* indique à votre ordinateur que Microsoft Word peut ouvrir le fichier et afficher une icône Word lorsque vous l'affichez dans l'Explorateur de fichiers.

- On va faire un petit essai. Je vous invite à écrire ce qui vous passe par la tête, comme moi à la figure suivante.



- Ouvrez maintenant l'explorateur de fichiers dans le dossier où vous avez enregistré votre page
- L'icône qui représente le fichier dépend de votre navigateur web par défaut, en général. Vous pouvez y voir une icône de Firefox, de Chrome...



- Le texte s'affiche sur la même ligne alors qu'on avait demandé à l'écrire sur deux lignes différentes. Que se passe-t-il ?
- En fait, pour créer une page web, il ne suffit pas de taper simplement du texte comme on vient de le faire. En plus de ce texte, il faut aussi écrire ce qu'on appelle des **balises**, qui vont donner des instructions à l'ordinateur comme « aller à la ligne », « afficher une image », etc

Les balises et leurs attributs

- Les pages HTML sont remplies de ce qu'on appelle des balises. Celles-ci sont invisibles à l'écran pour vos visiteurs, mais elles permettent à l'ordinateur de comprendre ce qu'il doit afficher.
- Les balises se repèrent facilement. Elles sont entourées de « chevrons », c'est-à-dire des symboles < et > , comme ceci : <balise> .
- On distingue deux types de balises : les balises en paires et les balises orphelines.
- Les balise en paires
 - Elles s'ouvrent, contiennent du texte, et se ferment plus loin. Voici à quoi elles ressemblent

```
<titre>Ceci est un titre</titre>
```

- On distingue une balise ouvrante (<titre>) et une balise fermante (</titre>) qui indique que le titre se termine. Cela signifie pour l'ordinateur que tout ce qui n'est pas entre ces deux balises... n'est pas un titre.

```
Ceci n'est pas un titre <titre>Ceci est un titre</titre> Ceci n'est pas un titre
```


- Les balises orphelines
 - Ce sont des balises qui servent le plus souvent à insérer un élément à un endroit précis (par exemple une image). Il n'est pas nécessaire de délimiter le début et la fin de l'image, on veut juste dire à l'ordinateur « Insère une image ici ».
 - Une balise orpheline s'écrit comme ceci :

```
<image />
```

Les attributs

- Les attributs sont un peu les options des balises. Ils viennent les compléter pour donner des informations supplémentaires. L'attribut se place après le nom de la balise ouvrante et a le plus souvent une valeur, comme ceci :

```
<balise attribut="valeur">
```

STRUCTURE DE BASE D'UNE PAGE HTML5

- Reprenons notre éditeur de texte Je vous invite à écrire le code source ci-dessous dans votre éditeur de texte. Ce code correspond à la base d'une page web en HTML5 :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>Titre</title>
  </head>

  <body>

  </body>
</html>
```

Vous noterez que les balises s'ouvrent et se ferment dans un ordre précis. Par exemple, la balise `<html>` est la première que l'on ouvre et c'est aussi la dernière que l'on ferme (tout à la fin du code, avec `</html>`). Les balises doivent être fermées dans le sens inverse de leur ouverture. Un exemple :

`<html><body></body></html>` : correct. Une balise qui est ouverte à l'intérieur d'une autre doit aussi être fermée à l'intérieur.

`<html><body></html></body>` : incorrect, les balises s'entremêlent.

Le doctype

- La toute première ligne s'appelle le **doctype**. Elle est indispensable car c'est elle qui indique qu'il s'agit bien d'une page web HTML.
Ce n'est pas vraiment une balise comme les autres (elle commence par un point d'exclamation). Vous pouvez considérer que c'est un peu l'exception qui confirme la règle.

La balise </html>

- C'est la balise principale du code. Elle englobe tout le contenu de votre page. Comme vous pouvez le voir, la balise fermante </html> se trouve tout à la fin du code !

```
<html>
```

```
</html>
```

L'encodage (charset)

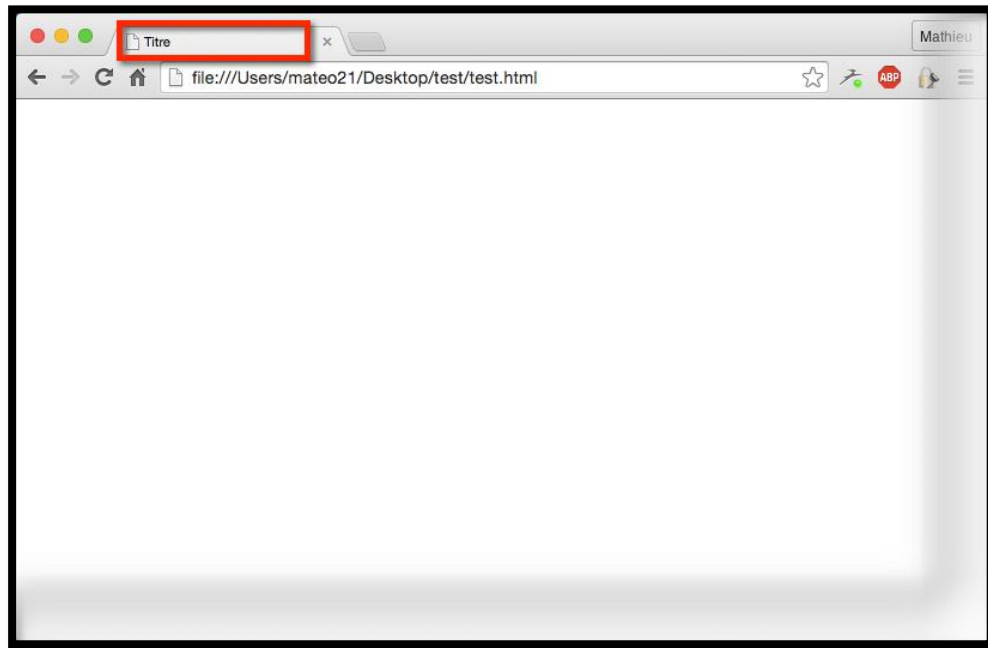
- Cette balise indique l'encodage utilisé dans votre fichier .html
- L'encodage indique la façon dont le fichier est enregistré. C'est lui qui détermine comment les caractères spéciaux vont s'afficher (accents, idéogrammes chinois et japonais, caractères arabes, etc.)..

L'en-tête <head> et le corps <body>

- Une page web est constituée de deux parties :
- L'en-tête <head> : cette section donne quelques informations générales sur la page, comme son titre, l'encodage (pour la gestion des caractères spéciaux), etc. Cette section est généralement assez courte. Les informations que contient l'en-tête ne sont pas affichées sur la page, ce sont simplement des informations générales à destination de l'ordinateur. Elles sont cependant très importantes !
- Le corps <body> : c'est là que se trouve la partie principale de la page. Tout ce que nous écrirons ici sera affiché à l'écran. C'est à l'intérieur du corps que nous écrirons la majeure partie de notre code.
- Pour le moment, le corps est vide (nous y reviendrons plus loin). Intéressons-nous par contre aux deux balises contenues dans l'en-tête...

■ Le titre principal de la page

- <title>
- C'est le titre de votre page, probablement l'élément le plus important ! Toute page doit avoir un titre qui décrit ce qu'elle contient.
- Le titre ne s'affiche pas dans votre page mais en haut de celle-ci (souvent dans l'onglet du navigateur). Enregistrez votre page web et ouvrez-la dans votre navigateur. Vous verrez que le titre s'affiche dans l'onglet, comme sur la figure suivante.



LES COMMENTAIRES

- Un **commentaire** en HTML est un texte qui sert simplement de mémo. Il n'est pas affiché, il n'est pas lu par l'ordinateur, cela ne change rien à l'affichage de la page.
- Cela sert à *vous* et aux personnes qui liront le code source de votre page. Vous pouvez utiliser les commentaires pour laisser des indications sur le fonctionnement de votre page.
- Un commentaire est une balise HTML avec une forme bien spéciale :

```
<!-- Ceci est un commentaire -->
```


LES PARAGRAPHES

- La plupart du temps, lorsqu'on écrit du texte dans une page web, on le fait à l'intérieur de paragraphes. Le langage HTML propose justement la balise `<p>` pour délimiter les paragraphes.

```
<p>Bonjour et bienvenue sur mon site !</p>
```

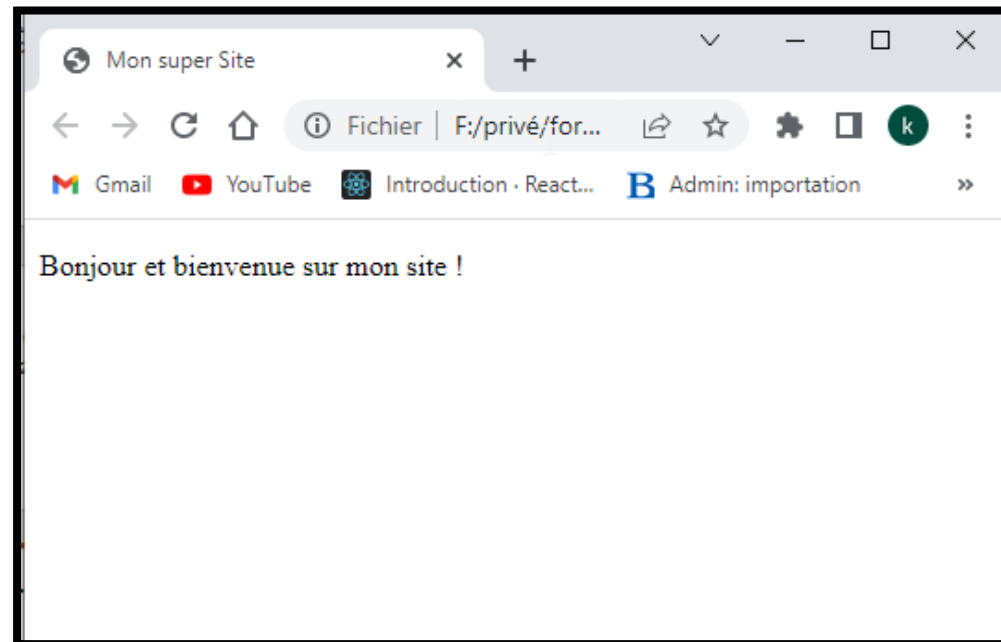
- `<p>` signifie « Début du paragraphe » ;
- `</p>` signifie « Fin du paragraphe ».
- Comme je vous l'ai dit au chapitre précédent, on écrit le contenu du site web entre les balises `<body></body>` . Il nous suffit donc de mettre notre paragraphe entre ces deux balises et nous aurons enfin notre première vraie page web avec du texte !

```
<!DOCTYPE html>
<html lang="fr">

<head>
  <meta charset="UTF-8">
  <title>Mon super Site</title>
</head>

<body>
  <p> Bonjour et bienvenue sur mon site ! </p>
</body>

</html>
```



Sauter la ligne

- En HTML, si vous appuyez sur la touche Entrée , cela ne crée pas une nouvelle ligne comme vous en avez l'habitude. Tout le texte s'affiche sur la même ligne alors qu'on est bien allé à la ligne dans le code ! Taper frénétiquement sur la touche Entrée dans l'éditeur de texte ne sert donc strictement à rien.
- En fait, si vous voulez écrire un deuxième paragraphe, il vous suffit d'utiliser une deuxième balise <p>

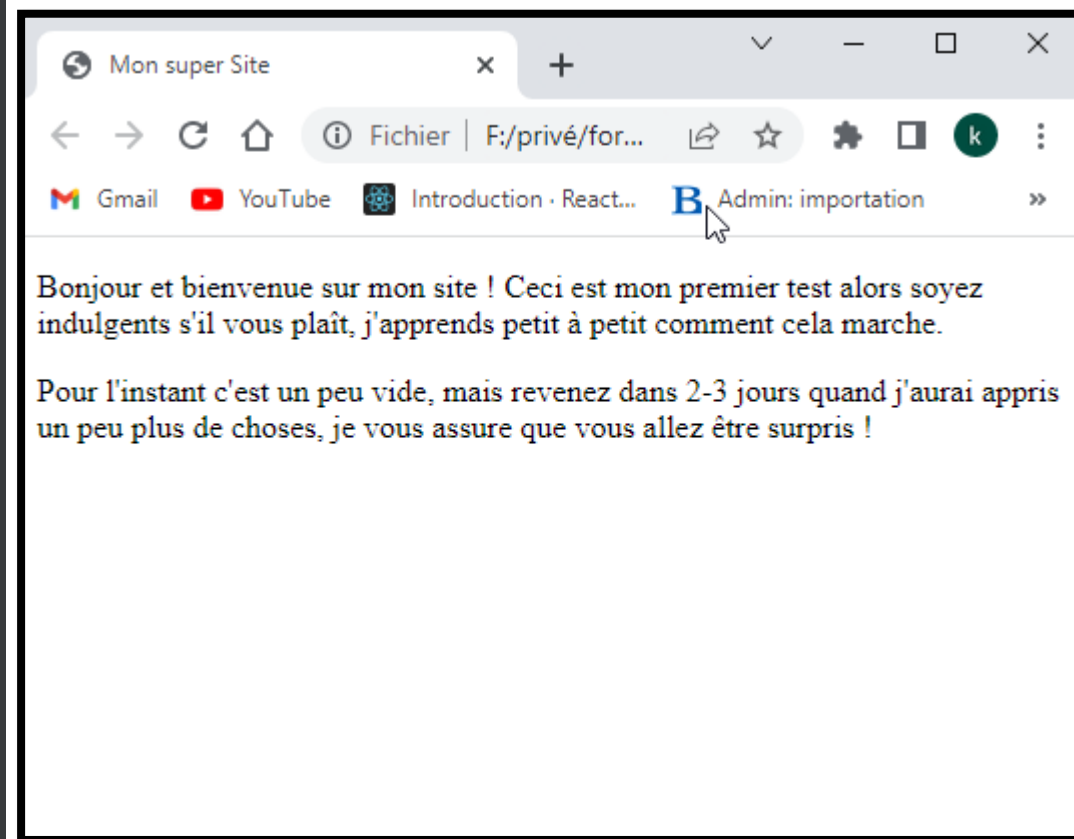
```
<!DOCTYPE html>
<html lang="fr">

<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Mon super Site</title>
</head>

<body>
  <p>Bonjour et bienvenue sur mon site ! Ceci est mon premier test
alors soyez indulgents s'il vous plaît, j'apprends
  petit à petit comment cela marche.</p>
  <p>Pour l'instant c'est un peu vide, mais revenez dans 2-3 jours
quand j'aurai appris un peu plus de choses, je vous
  assure que vous allez être surpris !</p>

</body>

</html>
```



LES TITRES

- Lorsque le contenu de votre page va s'étoffer avec de nombreux paragraphes, il va devenir difficile pour vos visiteurs de se repérer. C'est là que les titres deviennent utiles.
- En HTML, on est verni, on a le droit d'utiliser six niveaux de titres différents. Je veux dire par là qu'on peut dire « Ceci est un titre très important », « Ceci est un titre un peu moins important », « Ceci est un titre encore moins important », etc. On a donc six balises de titres différentes :
- `<h1> </h1>` : signifie « titre très important ». En général, on s'en sert pour afficher le titre de la page au début de celle-ci ;
- `<h2> </h2>` : signifie « titre important » ;
- `<h3> </h3>` : pareil, c'est un titre un peu moins important (on peut dire un « sous-titre », si vous voulez) ;
- `<h4> </h4>` : titre encore moins important ;
- `<h5> </h5>` : titre pas important ;
- `<h6> </h6>` : titre vraiment, mais alors là vraiment pas important du tout.

```

<!DOCTYPE html>
<html lang="fr">

<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mon super Site</title>
</head>

<body>
  <h1>Bienvenue sur notre ecole !</h1>
  <p>Bonjour et bienvenue sur mon site : notre ecole.<br />
    Notre ecole, qu'est-ce que c'est ?</p>
  <h2>Des cours pour débutants</h2>
  <p>Notre ecole vous propose des cours (tutoriels) destinés aux débutants :
aucune connaissance n'est requise pour
lire ces cours !</p>
  <p>Vous pourrez ainsi apprendre, sans rien y connaître auparavant, à créer
un site web, à programmer, à construire
des mondes en 3D !</p>
  <h2>Une communauté active</h2>
  <p>Vous avez un problème, un élément du cours que vous ne comprenez pas ?
Vous avez besoin d'aide pour créer votre
site ?<br />
Rendez-vous sur les forums ! Vous y découvrirez que vous n'êtes pas le
seul dans ce cas et vous trouverez très
certainement quelqu'un qui vous aidera aimablement à résoudre votre
problème.</p>

</body>

</html>

```



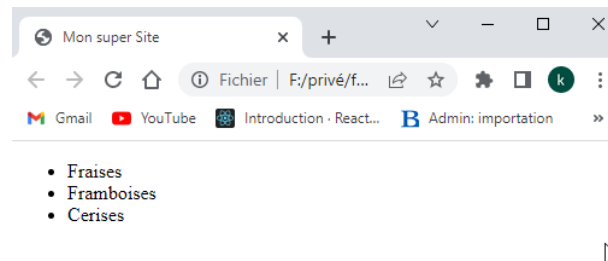
LES LISTES

- Les listes nous permettent souvent de mieux structurer notre texte et d'ordonner nos informations.
- Nous allons découvrir ici deux types de listes :
- les listes non ordonnées ou listes à puces ;
- les listes ordonnées ou listes numérotées, ou encore énumérations.

1, Liste non ordonnée

- C'est un système qui nous permet de créer une liste d'éléments sans notion d'ordre (il n'y a pas de « premier » ni de « dernier »). Créer une liste non ordonnée est très simple. Il suffit d'utiliser la balise `<ul>` que l'on referme un peu plus loin avec `</ul>`
- Commencez donc à taper ceci : `<ul></ul>`
- Et maintenant, voilà ce qu'on va faire : on va écrire chacun des éléments de la liste entre deux balises `<li></li>` . Chacune de ces balises doit se trouver entre `<ul>` et `</ul>` . Vous allez comprendre de suite avec cet exemple :

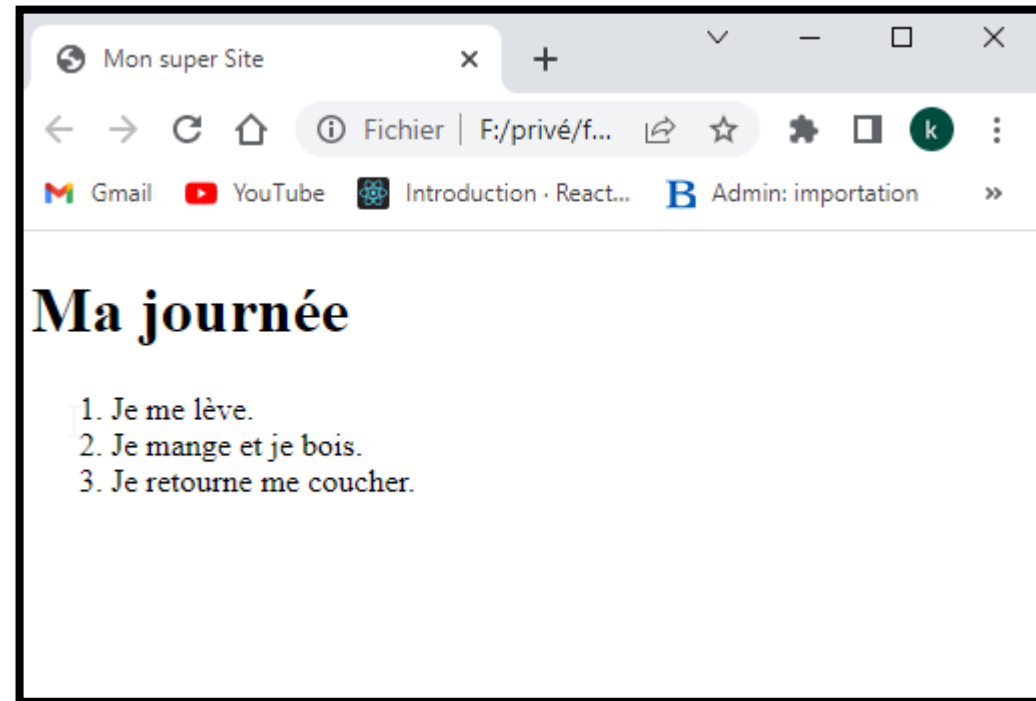
```
<body>
  <ul>
    <li>Fraises</li>
    <li>Framboises</li>
    <li>Cerises</li>
  </ul>
</body>
```



Liste ordonnée

- Une liste ordonnée fonctionne de la même façon, seule une balise change : il faut remplacer `<ul></ul>` par `<ol></ol>` .
- À l'intérieur de la liste, on ne change rien : on utilise toujours des balises `<li></li>` pour délimiter les éléments.

```
<body>
  <h1>Ma journée</h1>
  <ol>
    <li>Je me lève.</li>
    <li>Je mange et je bois.</li>
    <li>Je retourne me coucher.</li>
  </ol>
</body>
```



UN LIEN VERS UN AUTRE SITE

Pour faire un lien, la balise que nous allons utiliser est très simple à retenir : `<a>` . Il faut cependant lui ajouter un attribut, `href` , pour indiquer vers quelle page le lien doit conduire.

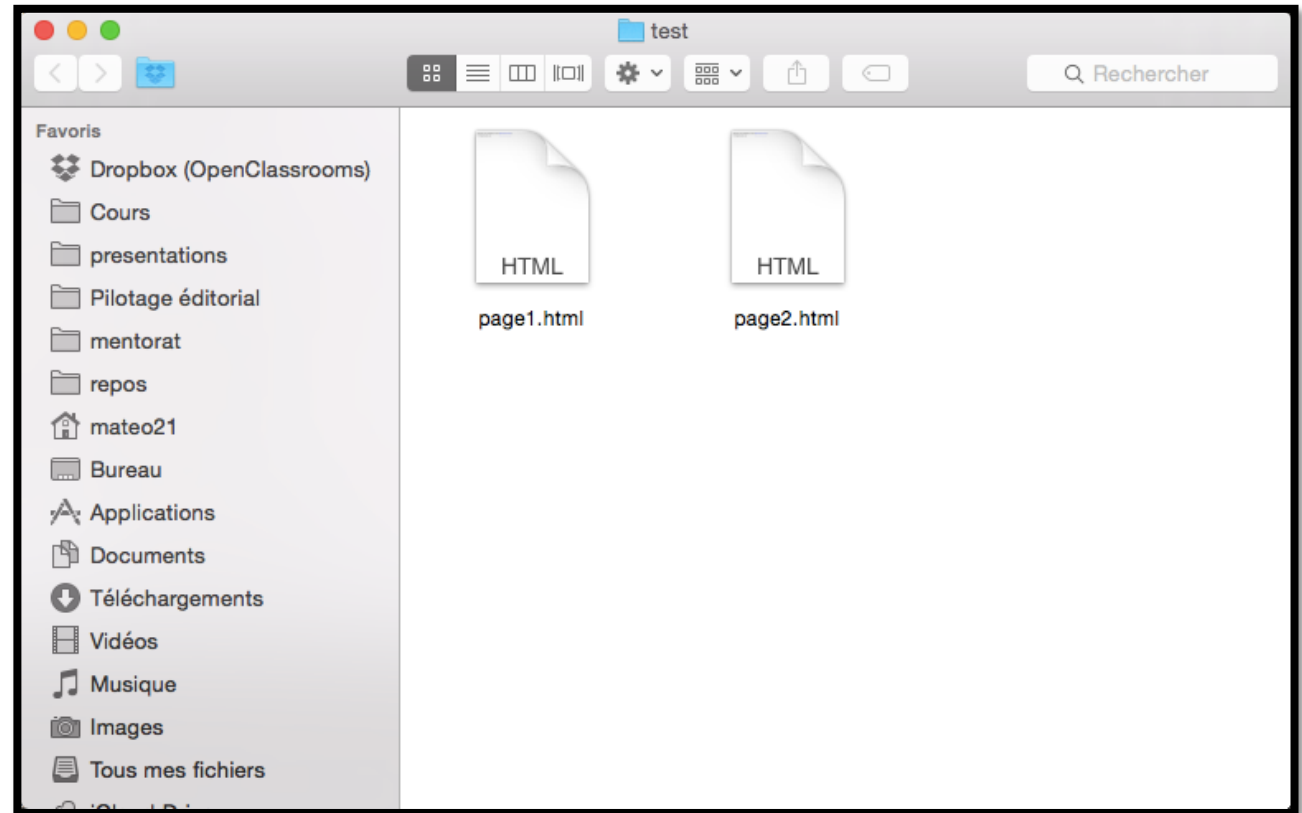
Par exemple, le code ci-dessous est un lien qui amène vers google, situé à l'adresse : <https://google.com>

```
<body>
  <p>Bonjour. Vous souhaitez visiter <a href="https://google.com">google</a> ?<br />
    C'est un bon site pour vos recherche!</p>
</body>
```

UN LIEN VERS UNE AUTRE PAGE DE SON SITE

- Pour commencer, nous allons créer deux fichiers correspondant à deux pages HTML différentes. Comme je suis très inspiré, je vous propose de les appeler `page1.html` et `page2.html`. Nous aurons donc ces deux fichiers sur notre disque dans le même dossier (figure suivante).

si les deux fichiers sont situés dans le même dossier, il suffit d'écrire comme cible du lien le nom du fichier vers lequel on veut amener. Par exemple : `<a href="page2.html">`. On dit que c'est un lien relatif.



- Voici le code que nous allons utiliser dans nos fichiers page1.html et page2.html .

- page1.html

```
<p>Bonjour. Souhaitez-vous consulter <a href="page2.html">la page 2</a> ?</p>
```

- page2.html

- La page 2 (page d'arrivée) affichera simplement un message pour indiquer que l'on est bien arrivé sur la page 2 :

```
<h1>Bienvenue sur la page 2 !</h1>
```

DEUX PAGES SITUÉES DANS DES DOSSIERS DIFFÉRENTS

- Imaginons que page2.html se trouve dans un sous-dossier appelé contenu , comme à la figure suivante.

Dans ce cas de figure, le lien doit être rédigé comme ceci :

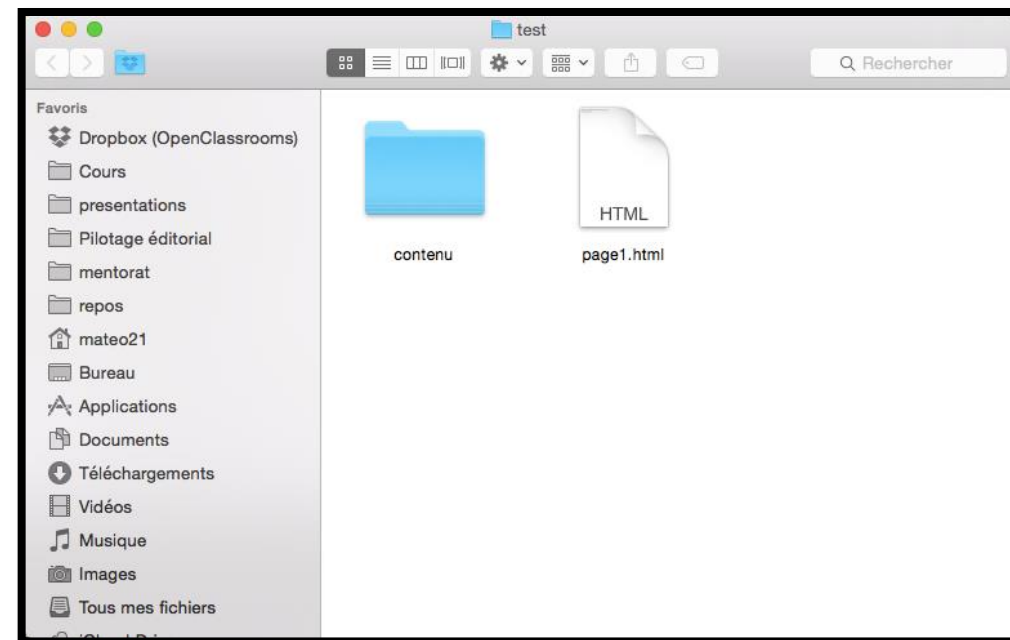
```
<a href="contenu/page2.html">
```

S'il y avait plusieurs sous-dossiers, on écrirait ceci :

```
<a href="contenu/autredossier/page2.html">
```

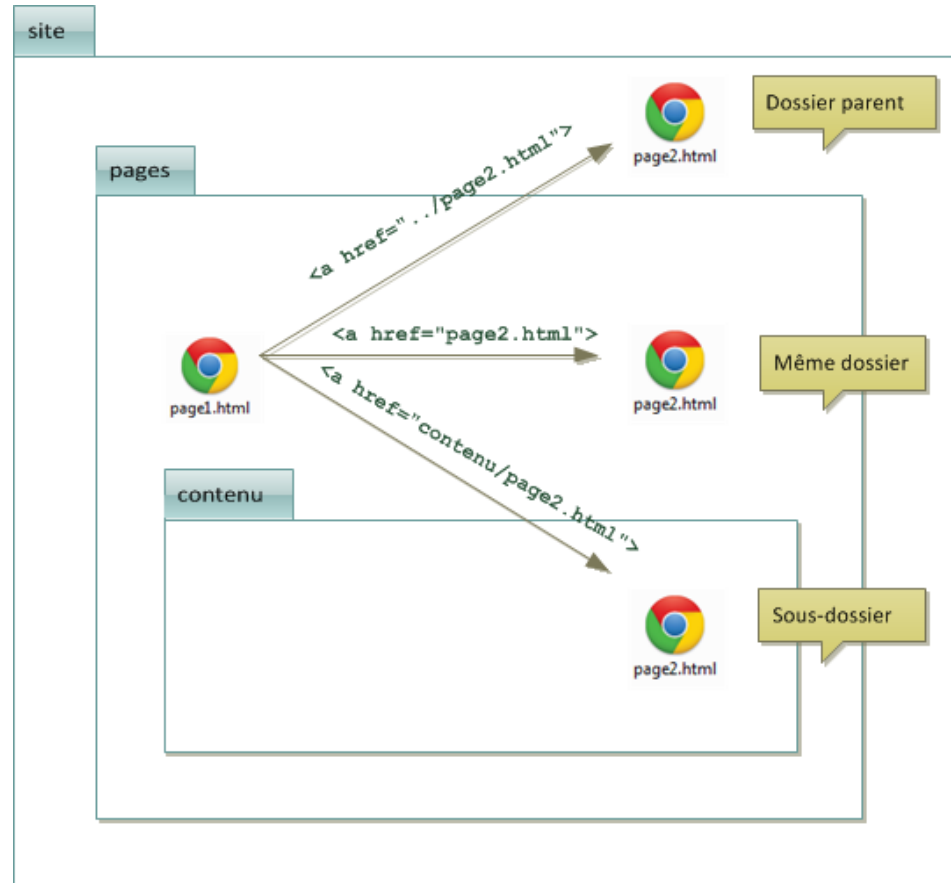
Si votre fichier cible est placé dans un dossier qui se trouve « plus haut » dans l'arborescence, il faut écrire deux points, comme ceci :

```
<a href="../page2.html">
```



RÉSUMÉ EN IMAGES

■ Les liens relatifs ne sont pas bien compliqués à utiliser une fois qu'on a compris le principe. Il suffit de regarder dans quel « niveau de dossier » se trouve votre fichier cible, pour savoir comment écrire votre lien. La figure suivante fait la synthèse des différents liens relatifs possibles.



INSÉRER UNE IMAGE

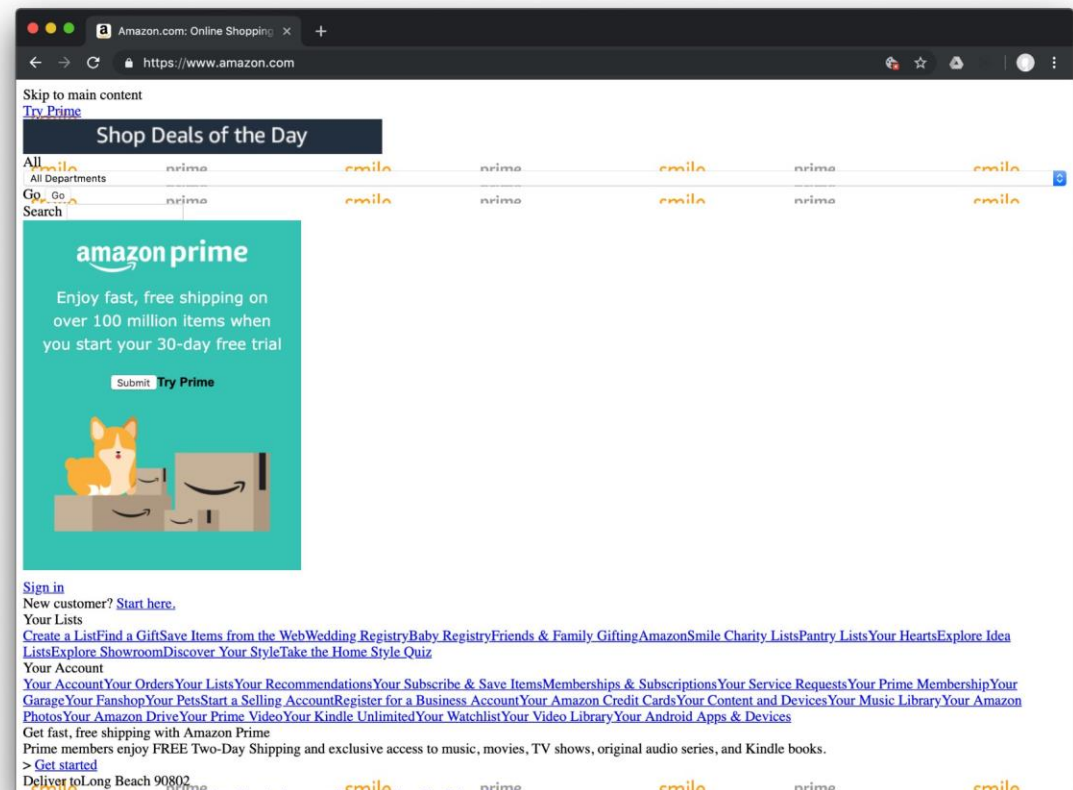
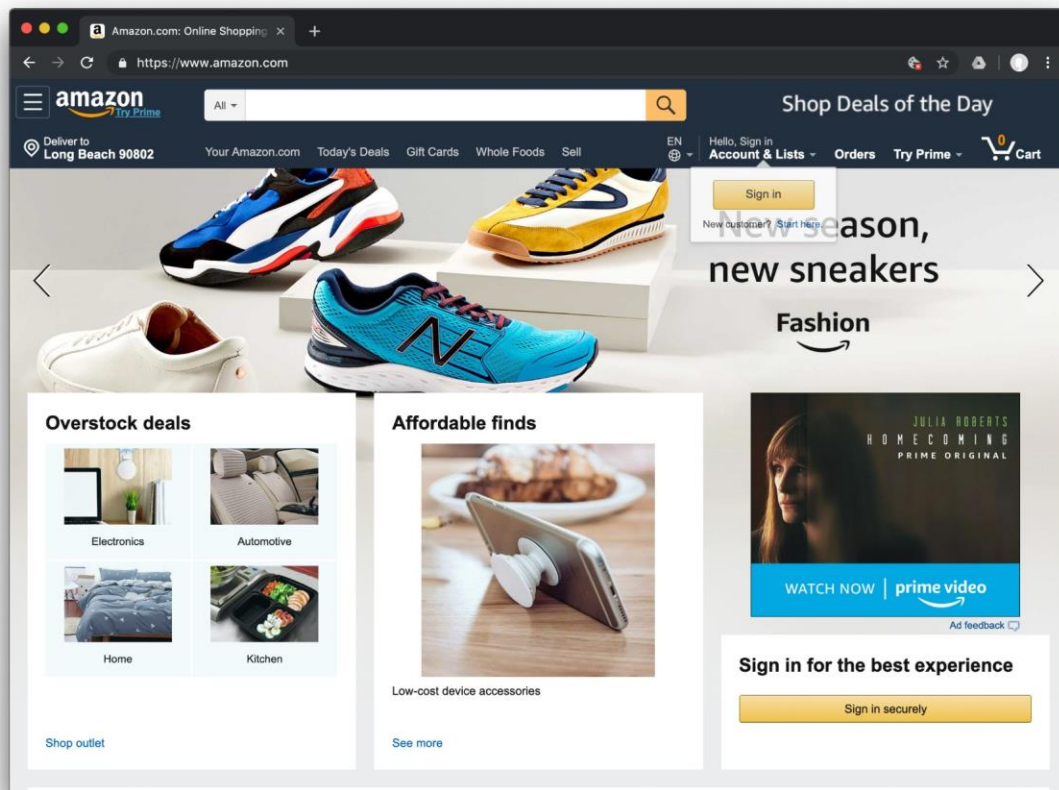
- Quelle est la fameuse balise qui va nous permettre d'insérer une image ? Il s'agit de... `<img />` !
- C'est une balise orpheline (comme `<br />`). Cela veut dire qu'on n'a pas besoin de l'écrire en deux exemplaires, comme la plupart des autres balises que nous avons vues jusqu'ici. En effet, nous n'avons pas besoin de délimiter une portion de texte, nous voulons juste insérer une image à un endroit précis.
- La balise doit être accompagnée de deux attributs obligatoires :
- `src` : il permet d'indiquer où se trouve l'image que l'on veut insérer. Vous pouvez soit mettre un chemin absolu (ex. : `http://www.site.com/fleur.png`), soit mettre le chemin en relatif (ce qu'on fait le plus souvent). Ainsi, si votre image est dans un sous-dossier `images` , vous devrez taper : `src="images/fleur.png"`
- `alt` : cela signifie « texte alternatif ». On doit toujours indiquer un texte alternatif à l'image, c'est-à-dire un court texte qui décrit ce que contient l'image. Ce texte sera affiché à la place de l'image si celle-ci ne peut pas être téléchargée (cela arrive), ou dans les navigateurs de personnes handicapées (non-voyants) qui ne peuvent malheureusement pas « voir » l'image. Cela aide aussi les robots des moteurs de recherche pour les recherches d'images. Pour la fleur, on mettrait par exemple : `alt="Une fleur"`
- Les images doivent se trouver obligatoirement à l'intérieur d'un paragraphe (`<p></p>`). Voici un exemple d'insertion d'image :

```
<p>
Voici une photo que j'ai prise lors de mes dernières vacances à la montagne :<br />

</p>
```

- Mettez en place le CSS
- Formatez du texte
- Ajoutez de la couleur et un fond
- Créez des bordures et des ombres
- Créez des apparences dynamiques

CSS



OÙ ÉCRIT-ON LE CSS ?

- Vous avez le choix car on peut écrire du code en langage CSS à trois endroits différents :
- dans un fichier .css (méthode la plus recommandée) ;
- dans l'en-tête <head> du fichier HTML ;
- directement dans les balises du fichier HTML via un attribut style (méthode la moins recommandée).

DANS UN FICHER .CSS (RECOMMANDÉ)

- C'est la méthode la plus pratique et la plus souple. Cela nous évite de tout mélanger dans un même fichier. J'utiliserai cette technique dans toute la suite de ce cours.
- Commençons à pratiquer dès maintenant ! Nous allons partir du fichier HTML suivant :

Vous noterez le contenu de la ligne 5, `<link rel="stylesheet" href="style.css" />` : c'est elle qui indique que ce fichier HTML est associé à un fichier appelé `style.css` et chargé de la mise en forme.

Enregistrez ce fichier sous le nom que vous voulez (par exemple, `page.html`). Pour le moment, rien d'extraordinaire à part la nouvelle balise que nous avons ajoutée.

```
<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8" />
  <link rel="stylesheet" href="style.css" />
  <title>Premiers tests du CSS</title>
</head>

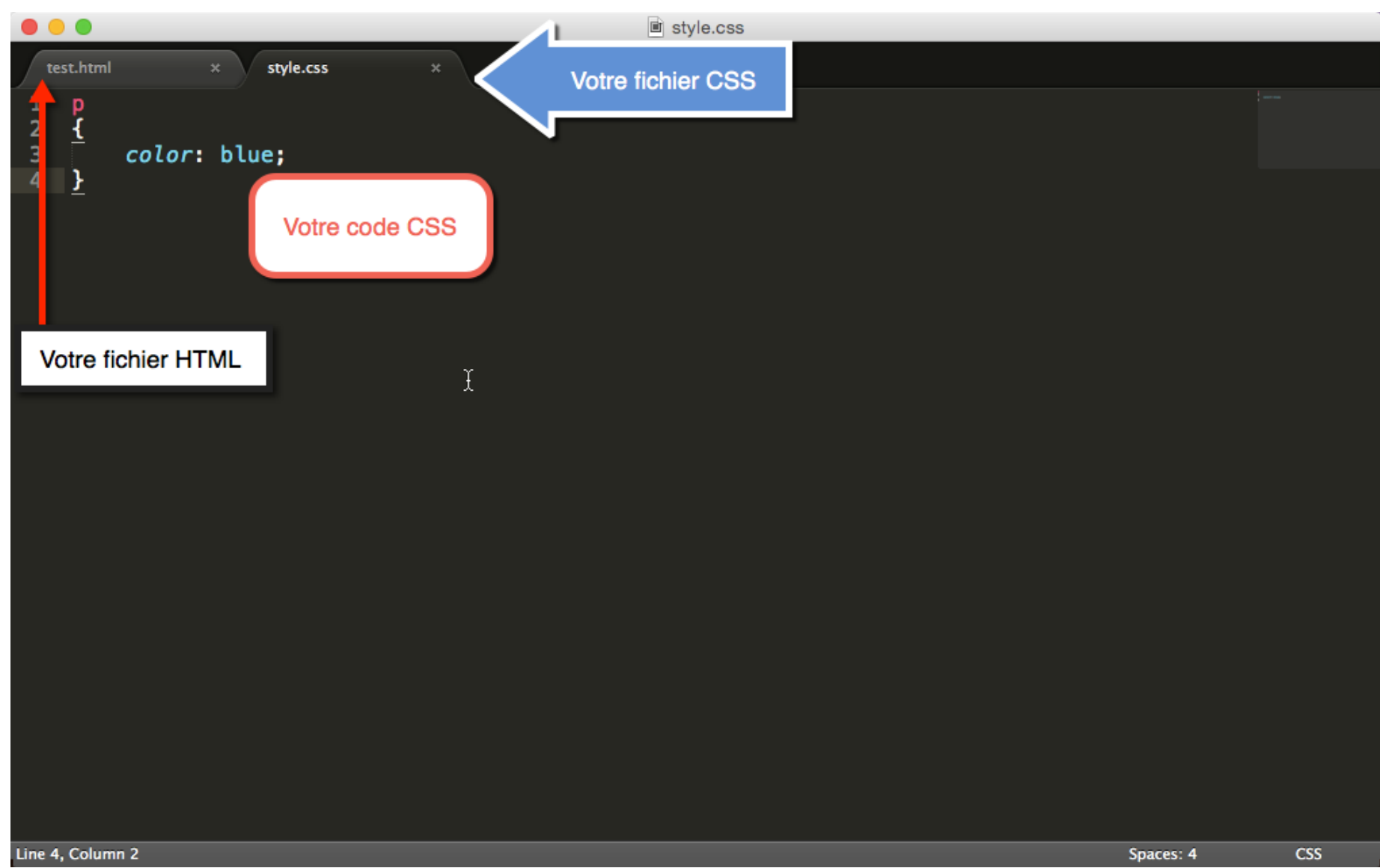
<body>
  <h1>Mon super site</h1>
  <p>Bonjour et bienvenue sur mon site !</p>
  <p>Pour le moment, mon site est un peu <em>vide</em>.
  Patientez encore un peu !</p>
</body>

</html>
```

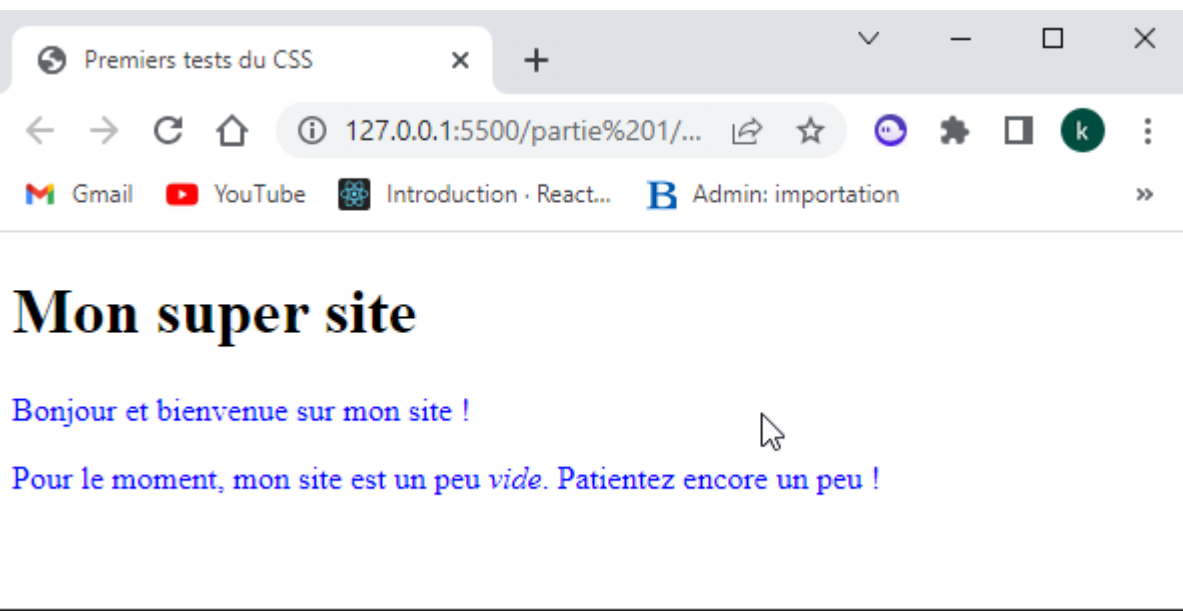
- Maintenant, créez un *nouveau* fichier vide dans votre éditeur de texte (par exemple Sublime Text) et copiez-y ce bout de code CSS (rassurez-vous, je vous expliquerai tout à l'heure ce qu'il veut dire) :

```
p
{
  color: blue;
}
```

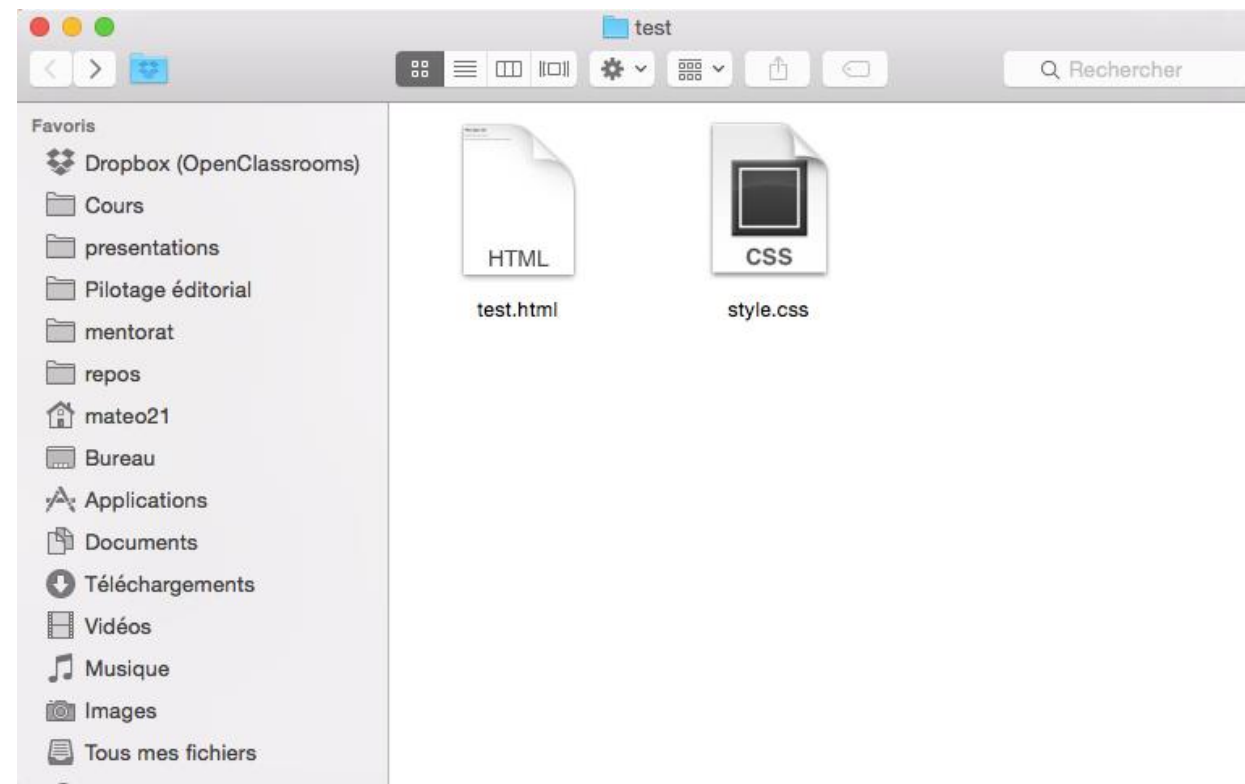
- Enregistrez le fichier en lui donnant un nom qui se termine par .css , comme style.css . Placez ce fichier .css dans le même dossier que votre fichier .html



- Ouvrez maintenant votre fichier page.html dans votre navigateur pour le tester, comme vous le faites d'habitude. Regardez, c'est magique : vos paragraphes sont écrits en bleu, comme dans la figure suivante !



- Dans votre explorateur de fichiers, vous devriez les voir apparaître côte à côte. D'un côté le .html , de l'autre le .css , comme à la figure suivante.



Notes:

Un fichier html peut appeler plusieurs fichiers css

Un fichier css peut être appelé par plusieurs fichiers html

L'appel au fichier css respecte la même logique que l'appel au lien relatif

DANS L'EN-TÊTE <HEAD> DU FICHER HTML

Dans l'en-tête <head> du fichier HTML

Il existe une autre méthode pour utiliser du CSS dans ses fichiers HTML : cela consiste à insérer le code CSS directement dans une balise <style> à l'intérieur de l'en-tête <head> .

Voici comment on peut obtenir exactement le même résultat avec un seul fichier .html qui contient le code CSS :

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8" />
<style>
p
{
  color: blue;
}
</style>
<title>Premiers tests du CSS</title>
</head>
<body>
<h1>Mon super site</h1>
<p>Bonjour et bienvenue sur mon site !</p>
<p>Pour le moment, mon site est un peu <em>vide</em>.
  Patientez encore un peu !</p>
</body>
</html>
```

DIRECTEMENT DANS LES BALISES (NON RECOMMANDÉ)

Dernière méthode, à manipuler avec précaution : vous pouvez ajouter un attribut style à n'importe quelle balise. Vous insérerez votre code CSS directement dans cet attribut :

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8" />
<title>Premiers tests du CSS</title>
</head>
<body>
<h1>Mon super site</h1>
<p style="color: blue;">Bonjour et bienvenue sur mon site
!</p>
<p>Pour le moment, mon site est un peu <em>vide</em>.
Patientez encore un peu !</p>
</body>
</html>
```

APPLIQUER UN STYLE : SÉLECTIONNER UNE BALISE

- Dans un code CSS, on trouve trois éléments différents :
- des noms de balises : on écrit les noms des balises dont on veut modifier l'apparence. Par exemple, si je veux modifier l'apparence de tous les paragraphes `<p>` , je dois écrire `p` ;
- des propriétés CSS : les « effets de style » de la page sont rangés dans des propriétés. Il y a par exemple la propriété `color` qui permet d'indiquer la couleur du texte, `font-size` qui permet d'indiquer la taille du texte, etc. Il y a beaucoup de propriétés CSS et, comme je vous l'ai dit, je ne vous obligerai pas à les connaître toutes par cœur ;
- les valeurs : pour chaque propriété CSS, on doit indiquer une valeur. Par exemple, pour la propriété `color` , il faut indiquer le nom de la couleur. Pour `font-size` , il faut indiquer quelle taille on veut, etc.

```
p
{
  color: blue;
}
```

- Schématiquement, une feuille de style CSS ressemble donc à cela :

```
balise1
{
propriete1: valeur1;
propriete2: valeur2;
propriete3: valeur3;
}
balise2
{
propriete1: valeur1;
propriete2: valeur2;
propriete3: valeur3;
propriete4: valeur4;
}
balise3
{
propriete1: valeur1;
}
```

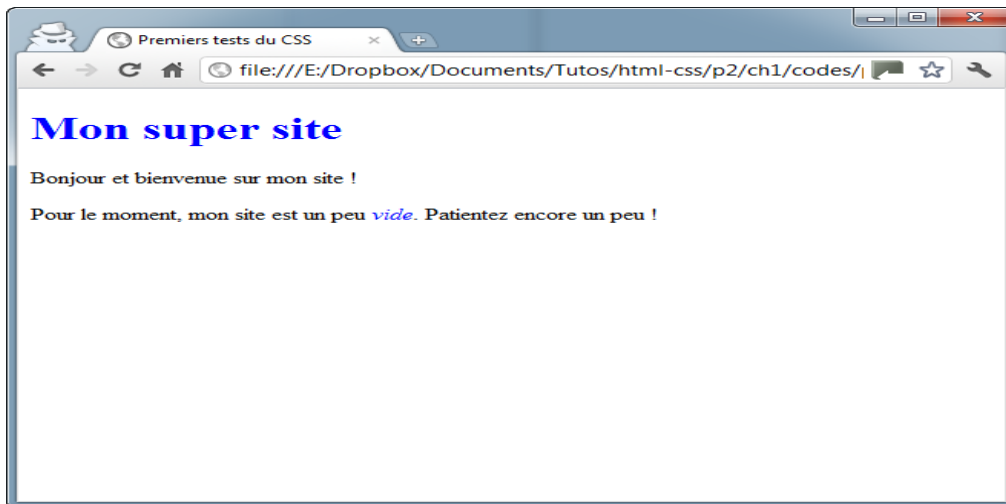
- Comme vous le voyez, on écrit le nom de la balise (par exemple h1) et on ouvre des accolades pour, à l'intérieur, mettre les propriétés et valeurs que l'on souhaite. On peut mettre autant de propriétés que l'on veut à l'intérieur des accolades. Chaque propriété est suivie du symbole « deux-points » (:) puis de la valeur correspondante. Enfin, chaque ligne se termine par un point-virgule (;).
- Je vous apprendrai de nombreuses propriétés dans les chapitres suivants. Pour le moment, dans les exemples, on va juste changer la couleur pour s'entraîner.
- Le code CSS que nous avons utilisé jusqu'ici :
- signifie donc en français : « *Je veux que tous mes paragraphes soient écrits en bleu.* ».

APPLIQUER UN STYLE À PLUSIEURS BALISES

- Prenons le code CSS suivant :
- Il signifie que nos titres `<h1>` et nos textes importants `<em>` doivent s'afficher en bleu. Par contre, c'est un peu répétitif,
- il existe un moyen en CSS d'aller plus vite si les deux balises doivent avoir la même présentation. Il suffit de combiner la déclaration en séparant les noms des balises par une virgule, comme ceci :

```
h1
{
  color: blue;
}
em
{
  color: blue;
}
```

```
h1, em
{
  color: blue;
}
```



APPLIQUER UN STYLE : CLASS ET ID

- Comment faire pour que certains paragraphes seulement soient écrits d'une manière différent
- Pour résoudre le problème, on peut utiliser ces attributs spéciaux qui fonctionnent sur toutes les balises :
- l'attribut class ;
- l'attribut id .
- les attributs class et id sont quasiment identiques. Il y a seulement une petite différence qu'on verra par la suite
- Pour le moment, et pour faire simple, on ne va s'intéresser qu'à l'attribut class .
- Comme je viens de vous le dire, c'est un attribut que l'on peut mettre sur n'importe quelle balise, aussi bien titre que paragraphe, image, etc.

```
<h1 class=""> </h1>  
<p class=""> </p>  
<img class="" />
```


- Par exemple, on va associer la classe introduction à notre premier paragraphe (ligne 12) :

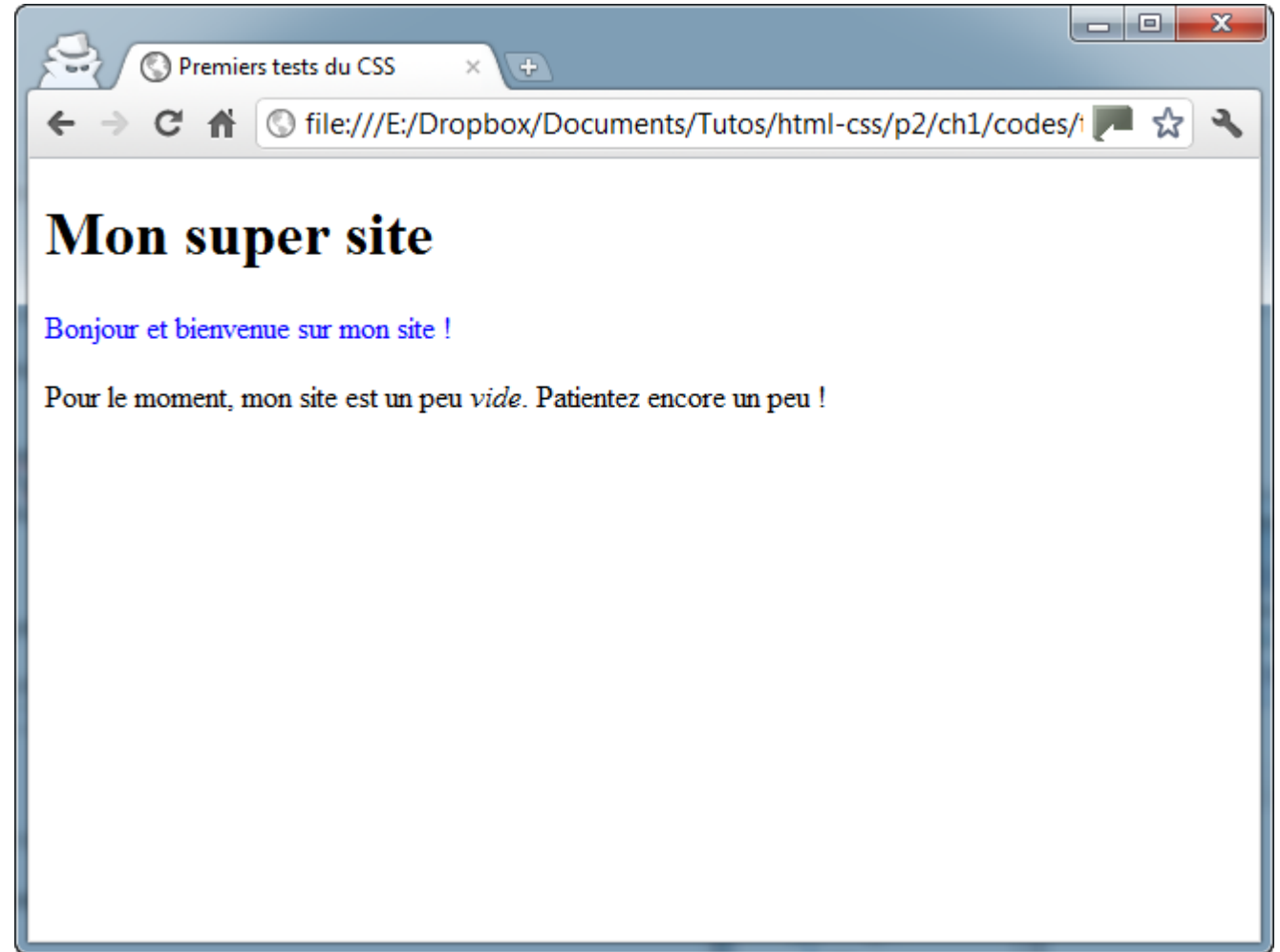
```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <meta charset="utf-8" />
5     <link rel="stylesheet" href="style.css" />
6     <title>Premiers tests du CSS</title>
7   </head>
8
9   <body>
10    <h1>Mon super site</h1>
11
12    <p class="introduction">Bonjour et bienvenue sur mon site !</p>
13    <p>Pour le moment, mon site est un peu <em>vide</em>. Patientez encore un peu !</p>
14  </body>
15 </html>
```

- Maintenant que c'est fait, votre paragraphe est identifié. Il a un nom : introduction . Vous allez pouvoir réutiliser ce nom dans le fichier CSS pour dire : « Je veux que seules les balises qui ont comme nom 'introduction' soient affichées en bleu ».
- Pour faire cela en CSS, indiquez le nom de votre classe en commençant par un point, comme ci-dessous faite le dans votre fichier css :

```
1 .introduction
2 {
3   color: blue;
4 }
```

- on sélectionne une class avec du css en mettant un point «.» comme prefixe au nom de la class

- Testez le résultat : seul votre paragraphe appelé introduction va s'afficher en bleu (figure suivante) !



- Id :
 - il fonctionne exactement de la même manière que class , à un détail près : il ne peut être utilisé qu'une fois dans le code.
 - ne mettons des id que sur des éléments qui sont uniques dans la page, comme par exemple le logo :
- ```

```
- Si vous utilisez des id , lorsque vous définirez leurs propriétés dans le fichier CSS, il faudra faire précéder le nom de l' id par un dièse (#) :

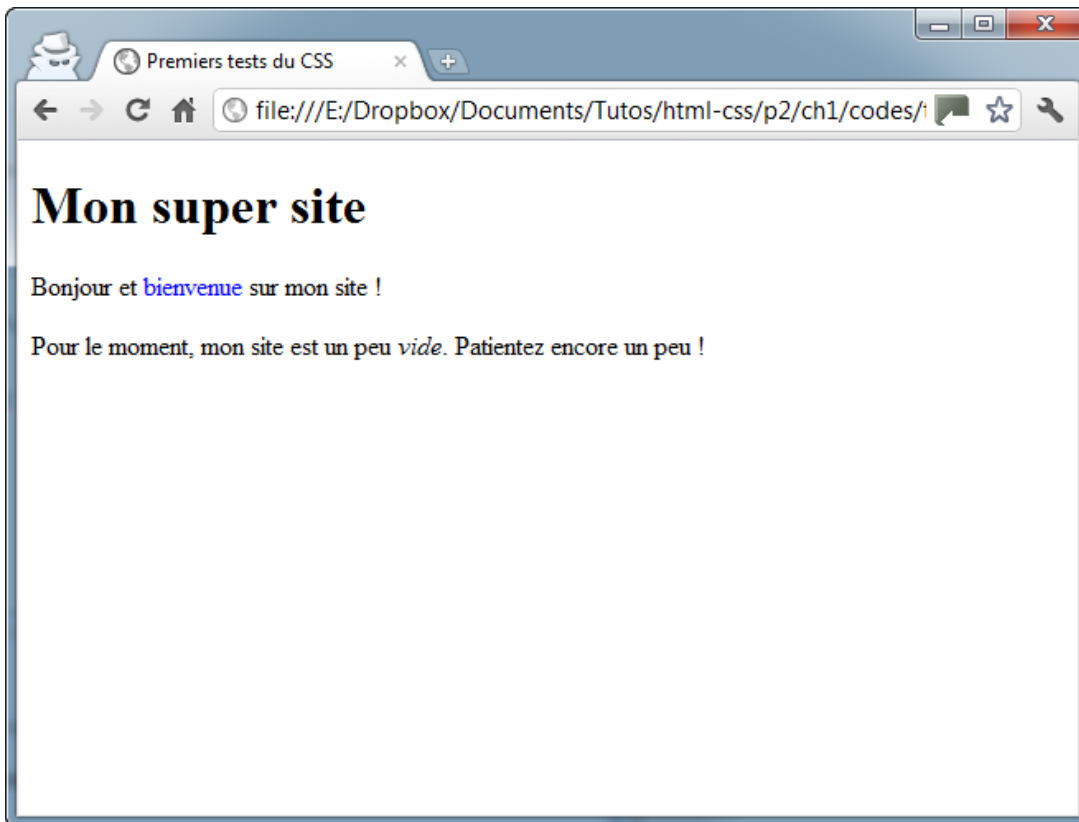
```
1 #logo
2 {
3 /* Indiquez les propriétés CSS ici */
4 }
4 }
```

# Les balises universelles

- Il arrivera parfois que vous ayez besoin d'appliquer une class (ou un id ) à certains mots qui, à l'origine, ne sont pas entourés par des balises.
- En effet, le problème de class , c'est qu'il s'agit d'un attribut. Vous ne pouvez donc en mettre que sur une balise. Si, par exemple, je veux modifier uniquement « bienvenue » dans le paragraphe suivant :  

```
<p>Bonjour et bienvenue sur mon site !</p>
```
- Cela serait facile à faire s'il y avait une balise autour de « bienvenue » , mais malheureusement, il n'y en a pas. Par chance, on a inventé... la balise-qui-ne-sert-à-rien.
- En fait, on a inventé deux balises dites universelles, qui n'ont aucune signification particulière (elles n'indiquent pas que le mot est important, par exemple). Il y a une différence minime (mais significative !) entre ces deux balises :
- `<span> </span>` : c'est une balise de type inline, c'est-à-dire une balise que l'on place au sein d'un paragraphe de texte pour sélectionner certains mots uniquement. Les balises `<strong>` et `<em>` sont de la même famille. Cette balise s'utilise donc au milieu d'un paragraphe et c'est celle dont nous allons nous servir pour colorer « bienvenue » ;

- `<div> </div>` : c'est une balise de type block, qui entoure un bloc de texte. Les balises `<p>` , `<h1>` , etc., sont de la même famille. Ces balises ont quelque chose en commun : elles créent un nouveau « bloc » dans la page et provoquent donc obligatoirement un retour à la ligne. `<div>` est une balise fréquemment utilisée dans la construction d'un design, comme nous le verrons plus tard.
- Pour le moment donc, nous allons utiliser plutôt la balise `<span>` . On la met autour de « bienvenue », on lui ajoute une classe (du nom qu'on veut), on crée le CSS et c'est gagné !



```
1 <p>Bonjour et bienvenue sur mon site !</p>
```

```
1 .salutations
2 {
3 color: blue;
4 }
```

# Formatez du texte : la taille

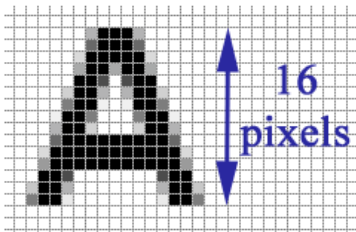
- Pour modifier la taille du texte, on utilise la propriété CSS `font-size` . Mais comment indiquer la taille du texte ? C'est là que les choses se corsent, car plusieurs techniques vous sont proposées :
- indiquer une taille absolue : en pixels, en centimètres ou millimètres. Cette méthode est très précise, mais il est conseillé de ne l'utiliser que si c'est absolument nécessaire, car on risque d'indiquer une taille trop petite pour certains lecteurs ;
- indiquer une taille relative : en pourcentage, « em » ou « ex », cette technique a l'avantage d'être plus souple. Elle s'adapte plus facilement aux préférences de taille des visiteurs.

# Une taille absolue

- Pour indiquer une taille absolue, on utilise généralement les pixels. Pour avoir un texte de 16 pixels de hauteur, vous devez donc écrire :

```
1 font-size: 16px;
```

Les lettres auront une taille de 16 pixels, comme le montre la figure suivante.



Une lettre de 16 pixels de hauteur

Voici un exemple d'utilisation (placez ce code dans votre fichier `.css`) :

```
1 p
2 {
3 font-size: 14px; /* Paragraphes de 14 pixels */
4 }
5 h1
6 {
7 font-size: 40px; /* Titres de 40 pixels */
8 }
```

CSS



# Une valeur relative

- C'est la méthode recommandée, car le texte s'adapte alors plus facilement aux préférences de tous les visiteurs.
- Il y a plusieurs moyens d'indiquer une valeur relative. Vous pouvez par exemple écrire la taille avec des mots en anglais comme ceux-ci :
- xx-small : minuscule ;
- x-small : très petit ;
- small : petit ;
- medium : moyen ;
- large : grand ;
- x-large : très grand ;
- xx-large : gigantesque.

```
p
{
 font-size: small;
}
h1
{
 font-size: large;
}
```



# LA POLICE

- La propriété CSS qui permet d'indiquer la police à utiliser est `font-family` . Vous devez écrire le nom de la police comme ceci :
- Voici une liste de polices qui fonctionnent bien sur la plupart des navigateurs :
- Arial ;
- Arial Black ;
- Comic Sans MS ;
- Courier New ;
- Georgia ;
- Impact ;
- Times New Roman ;
- Trebuchet MS ;
- Verdana.

```
balise
{
 font-family: police;
}
```

# L'alignement

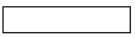
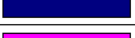
- Le langage CSS nous permet de faire tous les alignements connus : à gauche, centré, à droite et justifié.
- C'est tout simple. On utilise la propriété `text-align` et on indique l'alignement désiré :
- `left` : le texte sera aligné à gauche (c'est le réglage par défaut) ;
- `center` : le texte sera centré ;
- `right` : le texte sera aligné à droite ;
- `justify` : le texte sera « justifié ». Justifier le texte permet de faire en sorte qu'il prenne toute la largeur possible sans laisser d'espace blanc à la fin des lignes. Les textes des journaux, par exemple, sont toujours justifiés.
- Regardez les différents alignements sur cet exemple :

```
1 h1
2 {
3 text-align: center;
4 }
5
6 p
7 {
8 text-align: justify;
9 }
10
11 .signature
12 {
13 text-align: right;
14 }
```



# Ajoutez de la couleur et un fond: Couleur du texte

- Vous connaissez déjà la propriété qui permet de modifier la couleur du texte : il s'agit de `color` . Nous allons nous intéresser aux différentes façons d'indiquer la couleur, car il y en a plusieurs.
- Seize couleurs, c'est quand même un peu limite quand on sait que la plupart des écrans peuvent en afficher seize millions.
- la notation hexadécimale :
- Un nom de couleur en hexadécimal, cela ressemble à : `#FF5A28`. Pour faire simple, c'est une combinaison de lettres et de chiffres qui indiquent une couleur.
- On doit toujours commencer par écrire un dièse (`#`), suivi de six lettres ou chiffres allant de 0 à 9 et de A à F.
- Ces lettres ou chiffres fonctionnent deux par deux. Les deux premiers indiquent une quantité de rouge, les deux suivants une quantité de vert et les deux derniers une quantité de bleu. En mélangeant ces quantités (qui sont les composantes Rouge-Vert-Bleu de la couleur) on peut obtenir la couleur qu'on veut.
- Ainsi, `#000000` correspond à la couleur noire et `#FFFFFF` à la couleur blanche. Mais maintenant, ne me demandez pas quelle est la combinaison qui produit de l'orange couleur « coucher de soleil », je n'en sais strictement rien.
- Vous pouvez récupérer les code via google ^^

white	
silver	
gray	
black	
red	
maroon	
lime	
green	
yellow	
olive	
blue	
navy	
fuchsia	
purple	
aqua	
teal	

Voici par exemple comment on fait pour appliquer aux paragraphes la couleur blanche en hexadécimal :

```
1 p
2 {
3 color: #FFFFFF;
4 }
```

CSS

# Images de fond

- La propriété permettant d'indiquer une image de fond est `background-image` . Comme valeur, on doit renseigner `url("nom_de_l_image.png")` . Par exemple :

```
1 body
2 {
3 background-image: url("neige.png");
4 }
```

Ce qui nous donne la figure suivante.



# BORDURE

- Le CSS vous offre un large choix de bordures pour décorer votre page. De nombreuses propriétés CSS vous permettent de modifier l'apparence de vos bordures : `border-width` , `border-color` , `border-style`
- Pour aller à l'essentiel, je vous propose ici d'utiliser directement la super-propriété `border` qui regroupe l'ensemble de ces propriétés. Vous vous souvenez de la super-propriété `background` ? Cela fonctionne sur le même principe : on va pouvoir combiner plusieurs valeurs.
- Pour `border` , on peut utiliser jusqu'à trois valeurs pour modifier l'apparence de la bordure :
- la largeur : indiquez la largeur de votre bordure. Mettez une valeur en pixels (comme `2px` ) ;
- la couleur : c'est la couleur de votre bordure. Utilisez, comme on l'a appris, soit un nom de couleur ( `black` , `red` ...), soit une valeur hexadécimale ( `#FF0000` ), soit une valeur RGB ( `rgb(198, 212, 37)` ) ;

```
1 h1
2 {
3 border: 3px blue dashed;
4 }
```

Qui a dit que vous étiez obligé d'appliquer la même bordure aux quatre côtés de votre élément ?

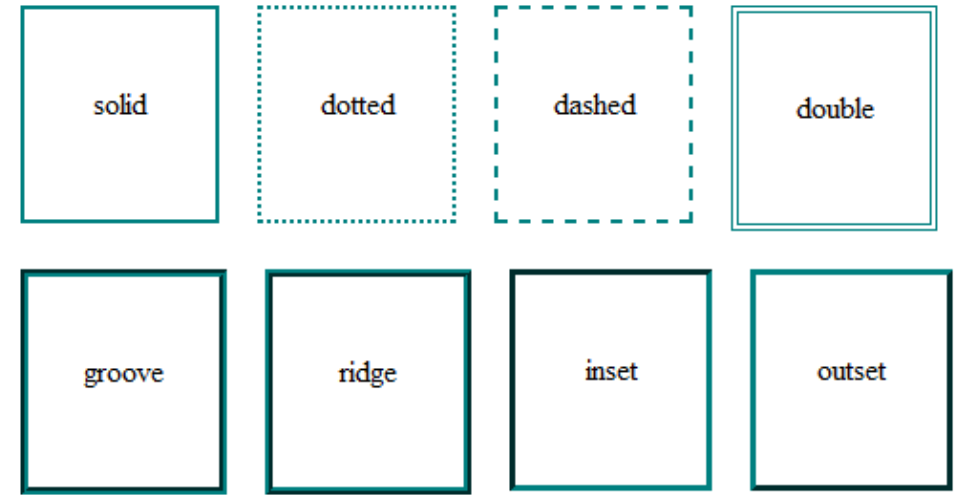
Taratata, si vous voulez mettre des bordures différentes en fonction du côté (haut, bas, gauche ou droite), vous pouvez le faire sans problème. Dans ce cas, vous devrez utiliser ces quatre propriétés :

`border-top` : bordure du haut ;

`border-bottom` : bordure du bas ;

`border-left` : bordure de gauche ;

`border-right` : bordure de droite.



```
1 p
2 {
3 border-left: 2px solid black;
4 border-right: 2px solid black;
5 }
```

Les bordures arrondies, c'est un peu le Saint Graal attendu par les webmasters depuis des millénaires (ou presque). Depuis que CSS3 est arrivé, il est enfin possible d'en créer facilement !

La propriété `border-radius` va nous permettre d'arrondir facilement les angles de n'importe quel élément. Il suffit d'indiquer la taille (« l'importance ») de l'arrondi en pixels :

```
1 p
2 {
3 border-radius: 10px;
4 }
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Cras ullamcorper sodales elit, sit amet pellentesque lectus aliquet quis. Etiam sem ipsum, rhoncus eu aliquam nec, mattis consectetur tortor. Mauris non lectus magna, vel interdum elit. Sed fermentum commodo commodo. Fusce imperdiet vestibulum neque, id pulvinar urna ultricies ullamcorper. Donec euismod, ipsum vehicula pretium tempor, mauris odio pellentesque metus, et ultrices arcu mauris sit amet leo. Curabitur ac scelerisque sem.

# LES OMBRES

- Les ombres font partie des nouveautés récentes proposées par CSS3. Aujourd'hui, il suffit d'une seule ligne de CSS pour ajouter des ombres dans une page !
- Nous allons ici découvrir deux types d'ombres :
  - les ombres des boîtes ;
  - les ombres du texte.
- `box-shadow` : les ombres des boîtes
- La propriété `box-shadow` s'applique à tout le bloc et prend quatre valeurs dans l'ordre suivant :
  - Le décalage horizontal de l'ombre.
  - Le décalage vertical de l'ombre.
  - L'adoucissement du dégradé.
  - La couleur de l'ombre.
- Par exemple, pour une ombre noire de 6 pixels, sans adoucissement, on écrira :

```
p {
 box-shadow: 6px 6px 0px black;
}
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Cras ullamcorper sodales elit, sit amet pellentesque lectus aliquet quis. Etiam sem ipsum, rhoncus eu aliquam nec, mattis consectetur tortor. Mauris non lectus magna, vel interdum elit. Sed fermentum commodo commodo. Fusce imperdiet vestibulum neque, id pulvinar urna ultricies ullamcorper. Donec euismod, ipsum vehicula pretium tempor, mauris odio pellentesque metus, et ultrices arcu mauris sit amet leo. Curabitur ac scelerisque sem.

```
p {
 text-shadow: 2px 2px 4px black;
}
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Cras ullamcorper sodales elit, sit amet pellentesque lectus aliquet quis. Etiam sem ipsum, rhoncus eu aliquam nec, mattis consectetur tortor. Mauris non lectus magna, vel interdum elit. Sed fermentum commodo commodo. Fusce imperdiet vestibulum neque, id pulvinar urna ultricies ullamcorper. Donec euismod, ipsum vehicula pretium tempor, mauris odio pellentesque metus, et ultrices arcu mauris sit amet leo. Curabitur ac scelerisque sem.



# MISE EN PAGE HTML CSS

Les techniques de mise en page des CSS nous permettent de prendre des éléments contenus dans une page web et d'en contrôler le placement par rapport à leur position par défaut dans le flot d'une mise en page courante, aux autres éléments environnants, à leur conteneur parents ou à la fenêtre principale d'affichage.

# DISPOSITION DES ÉLÉMENTS PAR DÉFAUT

Par défaut, le contenu d'un élément de niveau bloc prend 100% de la largeur de son élément parent et sa hauteur est commandée par son contenu. Les éléments en ligne ont la hauteur et la largeur de leur contenu. Vous ne pouvez pas définir la largeur ou la hauteur d'éléments en ligne – ils se trouvent uniquement à l'intérieur d'un contenu d'élément de niveau bloc. Si vous voulez contrôler la taille d'un élément en ligne de cette manière, vous devez le paramétrer pour qu'il se comporte comme un élément de niveau bloc avec `display: bloc` ; (ou même `display: inline-block` ; qui mélange les caractéristiques des deux).

```
<h2>Cours d'un document de base</h2>
```

```
<p>Je suis un élément de niveau bloc de base.
Mes éléments de niveau bloc adjacents sont sur de
nouvelles lignes en dessous de moi.</p>
```

```
<p>Par défaut, nous occupons 100% de la largeur
de notre élément parent et nous sommes aussi hauts
que notre contenu enfant. Nos largeur et hauteur totales
sont égales à la largeur/hauteur de notre
contenu + remplissage + encadrement.</p>
```

```
<p>Nous sommes séparés de nos marges.
Comme il y a fusion des marges, nous sommes séparés
par la largeur de l'une de nos marges et non les deux.</p>
```

```
<p>Des éléments inline comme celui-ci ou
celui-là sont placés sur la même ligne et
les nœuds de texte adjacents, s'il y a de la place sur
la même ligne. Les débordements des éléments inline
sont placés sur une nouvelle ligne si possible
(comme celle-ci contenant du texte), sinon ils
sont placés sur une nouvelle ligne, comme cette image

</p>
```

```
body {
 width: 500px;
 margin: 0 auto;
}

p {
 background: rgba(255,84,104,.3);
 border: 2px solid rgb(255,84,104);
 padding: 10px;
 margin: 10px;
}

span {
 background: white;
 border: 1px solid black;
}
```

## Cours d'un document de base

Je suis un élément de niveau bloc de base. Mes éléments de niveau bloc adjacents sont sur de nouvelles lignes en dessous de moi.

Par défaut, nous occupons 100% de la largeur de notre élément parent et nous sommes aussi hauts que notre contenu enfant. Nos largeur et hauteur totales sont égales à la largeur/hauteur de notre contenu + remplissage + encadrement.

Nous sommes séparés de nos marges. Comme il y a fusion des marges, nous sommes séparés par la largeur de l'une de nos marges et non les deux.

Des éléments inline comme celui-ci ou celui-là sont placés sur la même ligne et les nœuds de texte adjacents, s'il y a de la place sur la même ligne. Les débordements des éléments inline sont placés sur une nouvelle ligne si possible (comme celle-ci contenant du texte), sinon ils sont placés sur une nouvelle ligne, comme cette image : 

# FLEXBOX

Pendant longtemps, les seuls outils de mise en page CSS fiables et compatibles avec les navigateurs, étaient les propriétés concernant les flotteurs (boîtes flottantes) et le positionnement. Ces outils sont bien et fonctionnent, mais restent à certains égards plutôt limitatifs et frustrants.

Les simples exigences de mise en page suivantes sont difficiles sinon impossibles à réaliser de manière pratique et souple avec ces outils :

Centrer verticalement un bloc de contenu dans son parent ;

Faire que tous les enfants d'un conteneur occupent tous une même quantité de hauteur/largeur disponible selon l'espace offert ;

Faire que toutes les colonnes dans une disposition multi-colonnes aient la même hauteur même si leur quantité de contenu diffère.

Comme vous le verrez dans les paragraphes suivants, Flexbox facilite considérablement les tâches de mise en page. Approfondissons un peu !

Pour commencer, sélectionnons les éléments devant être présentés sous forme de boîtes flexibles. Pour ce faire, donnons une valeur spéciale à la propriété display du parent de ces éléments à disposer. Dans ce cas, comme cela concerne les éléments <article>, nous affectons la valeur flex à l'élément <section> (qui devient un conteneur flex) :

```
section {
 display: flex;
}
```

### Sample flexbox example

#### First article

Tacos actually microdosing, pour-over semiotics banjo chicharrones retro fanny pack portland everyday carry vinyl typewriter. Tacos PBR&B pork belly, everyday carry ennui pickled sriracha normcore hashtag polaroid single-origin coffee cold-pressed. PBR&B tattooed trust fund twee, leggings salvia iPhone photo booth health goth gastropub hammock.

#### Second article

Tacos actually microdosing, pour-over semiotics banjo chicharrones retro fanny pack portland everyday carry vinyl typewriter. Tacos PBR&B pork belly, everyday carry ennui pickled sriracha normcore hashtag polaroid single-origin coffee cold-pressed. PBR&B tattooed trust fund twee, leggings salvia iPhone photo booth health goth gastropub hammock.

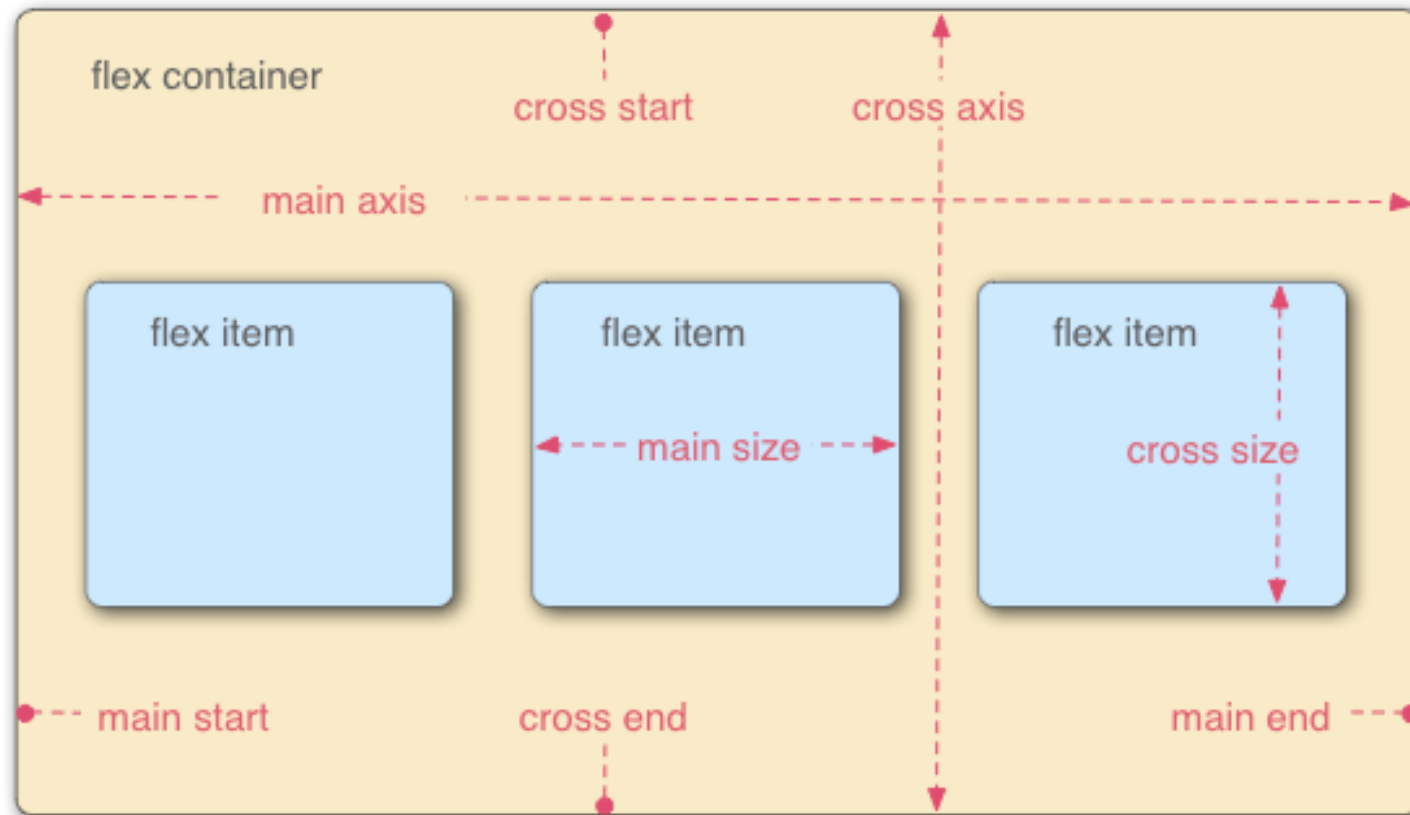
#### Third article

Tacos actually microdosing, pour-over semiotics banjo chicharrones retro fanny pack portland everyday carry vinyl typewriter. Tacos PBR&B pork belly, everyday carry ennui pickled sriracha normcore hashtag polaroid single-origin coffee cold-pressed. PBR&B tattooed trust fund twee, leggings salvia iPhone photo booth health goth gastropub hammock.

Cray food truck brunch, XOXO +1 keffiyeh pickled chambray waistcoat ennui. Organic small batch paleo 8-bit. Intelligentsia umami wayfarers pickled, asymmetrical kombucha letterpress kitsch leggings cold-pressed squid chartreuse put a bird on it. Listicle pickled man bun cornhole heirloom art party.

Lorsque les éléments sont disposés en boîtes flexibles, ils sont disposés le long de deux axes :

- L'axe principal (main axis) est l'axe de la direction dans laquelle sont disposés les éléments flex (par exemple, horizontalement sur la page, ou verticalement de haut en bas de la page). Le début et la fin de cet axe sont appelés l'origine principale (main start) et la fin principale (main end).
- L'axe croisé (cross axis) est l'axe perpendiculaire à l'axe principal, c'est-à-dire à la direction dans laquelle sont disposés les éléments flex. Le début et la fin de cet axe sont appelés le début (cross start) et la fin (cross end) de l'axe croisé.
- L'élément parent dont la propriété est `display: flex` (<section> dans notre exemple) est appelé le conteneur flex (flex container).
- Les éléments disposés en tant que boîtes flexibles à l'intérieur du conteneur flex sont appelés éléments flex (flex items) (les éléments <article> dans notre exemple).



Flexbox dispose de la propriété flex-direction pour indiquer la direction de l'axe principal (direction dans laquelle les enfants flexibles sont disposés). Cette propriété est égale par défaut à row : ils sont donc disposés en ligne, dans le sens de lecture de la langue par défaut du navigateur (de gauche à droite, dans le cas d'un navigateur français).

Ajoutez la déclaration suivante dans la règle CSS pour l'élément `<section>` :



```
flex-direction: column;
```

# Sample flexbox example

First article	Second article	Third article	Fourth article	Fifth article	Sixth article	Seventh article	Eighth article
Tacos actually microdosing, pour-over semiotics banjo chicharrones retro fanny pack portland everyday carry vinyl typewriter. Tacos PBR&B pork belly, everyday carry ennui pickled sriracha normcore hashtag polaroid single-origin coffee cold-pressed. PBR&B tattooed trust fund twee, leggings salvia iPhone photo booth	Tacos actually microdosing, pour-over semiotics banjo chicharrones retro fanny pack portland everyday carry vinyl typewriter. Tacos PBR&B pork belly, everyday carry ennui pickled sriracha normcore hashtag polaroid single-origin coffee cold-pressed. PBR&B tattooed trust fund twee, leggings salvia iPhone photo booth	Tacos actually microdosing, pour-over semiotics banjo chicharrones retro fanny pack portland everyday carry vinyl typewriter. Tacos PBR&B pork belly, everyday carry ennui pickled sriracha normcore hashtag polaroid single-origin coffee cold-pressed. PBR&B tattooed trust fund twee, leggings salvia iPhone photo booth	Tacos actually microdosing, pour-over semiotics banjo chicharrones retro fanny pack portland everyday carry vinyl typewriter. Tacos PBR&B pork belly, everyday carry ennui pickled sriracha normcore hashtag polaroid single-origin coffee cold-pressed. PBR&B tattooed trust fund twee, leggings salvia iPhone photo booth	Tacos actually microdosing, pour-over semiotics banjo chicharrones retro fanny pack portland everyday carry vinyl typewriter. Tacos PBR&B pork belly, everyday carry ennui pickled sriracha normcore hashtag polaroid single-origin coffee cold-pressed. PBR&B tattooed trust fund twee, leggings salvia iPhone photo booth	Tacos actually microdosing, pour-over semiotics banjo chicharrones retro fanny pack portland everyday carry vinyl typewriter. Tacos PBR&B pork belly, everyday carry ennui pickled sriracha normcore hashtag polaroid single-origin coffee cold-pressed. PBR&B tattooed trust fund twee, leggings salvia iPhone photo booth	Tacos actually microdosing, pour-over semiotics banjo chicharrones retro fanny pack portland everyday carry vinyl typewriter. Tacos PBR&B pork belly, everyday carry ennui pickled sriracha normcore hashtag polaroid single-origin coffee cold-pressed. PBR&B tattooed trust fund twee, leggings salvia iPhone photo booth	Tacos actually microdosing, pour-over semiotics banjo chicharrones retro fanny pack portland everyday carry vinyl typewriter. Tacos PBR&B pork belly, everyday carry ennui pickled sriracha normcore hashtag polaroid single-origin coffee cold-pressed. PBR&B tattooed trust fund twee, leggings salvia iPhone photo booth

quand votre structure est de largeur ou hauteur fixe, il arrive que les éléments flex débordent du conteneur et brisent cette structure.

```
flex-wrap: wrap;
```

Sample flexbox example

Fourth article

Tacos actually microdosing, pour-over semiotics banjo chicharrones retro fanny pack portland everyday carry vinyl typewriter. Tacos PBR&B pork belly, everyday carry ennui pickled sriracha normcore hashtag polaroid single-origin coffee cold-pressed. PBR&B tattooed trust fund twee, leggings salvia iPhone photo booth health goth gastropub hammock.

Third article

Tacos actually microdosing, pour-over semiotics banjo chicharrones retro fanny pack portland everyday carry vinyl typewriter. Tacos PBR&B pork belly, everyday carry ennui pickled sriracha normcore hashtag polaroid single-origin coffee cold-pressed. PBR&B tattooed trust fund twee, leggings salvia iPhone photo booth health goth gastropub hammock.  
  
Cray food truck brunch, XOXO +1 keffiyeh pickled chambray waistcoat ennui. Organic small batch paleo 8-bit. Intelligentsia umami wayfarers pickled, asymmetrical kombucha letterpress kitsch leggings cold-pressed squid chartreuse put a bird on it. Listic pickled man bun cornhole heirloom art party.

Second article

Tacos actually microdosing, pour-over semiotics banjo chicharrones retro fanny pack portland everyday carry vinyl typewriter. Tacos PBR&B pork belly, everyday carry ennui pickled sriracha normcore hashtag polaroid single-origin coffee cold-pressed. PBR&B tattooed trust fund twee, leggings salvia iPhone photo booth health goth gastropub hammock.

First article

Tacos actually microdosing, pour-over semiotics banjo chicharrones retro fanny pack portland everyday carry vinyl typewriter. Tacos PBR&B pork belly, everyday carry ennui pickled sriracha normcore hashtag polaroid single-origin coffee cold-pressed. PBR&B tattooed trust fund twee, leggings salvia iPhone photo booth health goth gastropub hammock.

Eigth article

Tacos actually microdosing, pour-over semiotics banjo

Seventh article

Tacos actually microdosing, pour-over semiotics banjo

Six article

Tacos actually microdosing, pour-over semiotics banjo

Fifth article

Tacos actually microdosing, pour-over semiotics banjo

Il existe plusieurs d'autre propriété flex je vous laisse le soin de les découvrir sur le site suivant

[https://www.w3schools.com/css/css3\\_flexbox.asp](https://www.w3schools.com/css/css3_flexbox.asp)

# LES BOÎTES FLOTTANTES

La propriété float a été introduite pour permettre aux développeurs web d'implémenter des dispositions simples comme une image flottant dans une colonne de texte, le texte se développant autour de cette image sur la gauche ou sur la droite. Le genre de choses que vous pourriez avoir dans une mise en page de journal.

```

<h1>Exemple simple de boîte flottante</h1>

<div class="box">Boîte flottante</div>

<p>
 Lorem ipsum dolor sit amet, consectetur adipiscing elit.
 Nulla luctus aliquam dolor, eu lacinia lorem placerat vulputate.
 Duis felis orci, pulvinar id metus ut, rutrum luctus orci.
 Cras porttitor imperdiet nunc, at ultricies tellus laoreet sit amet.
</p>

<p>
 Sed auctor cursus massa at porta. Integer ligula ipsum,
 tristique sit amet orci vel, viverra egestas ligula. Curabitur
 vehicula tellus neque, ac ornare ex malesuada et. In vitae
 convallis lacus. Aliquam erat volutpat. Suspendisse ac imperdiet
 turpis. Aenean finibus sollicitudin eros pharetra congue. Duis
 ornare egestas augue ut luctus. Proin blandit quam nec lacus
 varius commodo et a urna. Ut id ornare felis, eget fermentum sapien.
</p>

<p>
 Nam vulputate diam nec tempor bibendum. Donec luctus augue eget
 malesuada ultrices. Phasellus turpis est, posuere sit amet dapibus
 ut, facilisis sed est. Nam id risus quis ante semper consectetur
 eget aliquam lorem. Vivamus tristique elit dolor, sed pretium metus
 suscipit vel. Mauris ultricies lectus sed lobortis finibus. Vivamus
 eu urna eget velit cursus viverra quis vestibulum sem. Aliquam
 tincidunt eget purus in interdum. Cum sociis natoque penatibus et
 magnis dis parturient montes, nascetur ridiculus mus.
</p>

```

```

body {
 width: 90%;
 max-width: 900px;
 margin: 0 auto;
 font: .9em/1.2 Arial, Helvetica, sans-serif;
}

.box {
 float: left;
 margin-right: 15px;
 width: 150px;
 height: 150px;
 border-radius: 5px;
 background-color: rgb(207,232,220);
 padding: 1em;
}

```

## Exemple simple de boîte flottante

Boîte flottante

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla luctus aliquam dolor, eu lacinia lorem placerat vulputate. Duis felis orci, pulvinar id metus ut, rutrum luctus orci. Cras porttitor imperdiet nunc, at ultricies tellus laoreet sit amet.

Sed auctor cursus massa at porta. Integer ligula ipsum, tristique sit amet orci vel, viverra egestas ligula. Curabitur vehicula tellus neque, ac ornare ex malesuada et. In vitae convallis lacus. Aliquam erat volutpat. Suspendisse ac imperdiet turpis. Aenean finibus sollicitudin eros pharetra congue. Duis ornare egestas augue ut luctus. Proin blandit quam nec lacus varius commodo et a urna. Ut id ornare felis, eget fermentum sapien.

Nam vulputate diam nec tempor bibendum. Donec luctus augue eget malesuada ultrices. Phasellus turpis est, posuere sit amet dapibus ut, facilisis sed est. Nam id risus quis ante semper consectetur eget aliquam lorem. Vivamus tristique elit dolor, sed pretium metus suscipit vel. Mauris ultricies lectus sed lobortis finibus. Vivamus eu urna eget velit cursus viverra quis vestibulum sem. Aliquam tincidunt eget purus in interdum. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus.

# LE POSITIONNEMENT

Le positionnement permet de sortir les éléments du flux normal de la composition du document, et de les faire se comporter différemment, par exemple en plaçant un élément sur un autre ou en occupant toujours la même place dans la zone d'affichage du navigateur. Cet article explique les diverses valeurs de position, et comment les utiliser.

Le positionnement statique est celui reçu par défaut par chaque élément. Cela veut tout simplement dire « positionner l'élément selon le flux normal, rien de spécial à voir ici ».

Pour illustrer ce positionnement (et disposer d'exemple qui nous servira pour les prochaines sections), ajoutez tout d'abord une classe `positioned` pour le deuxième `<p>` dans le HTML :

```
<p class="positioned"> ... </p>
```

Puis ajoutez la règle suivante au bas de votre CSS :

```
.positioned {
 position: static;
 background: yellow;
}
```

Si vous sauvegardez et actualisez, vous verrez qu'il n'y a aucune différence, à l'exception de la mise à jour de la couleur de l'arrière-plan du deuxième paragraphe. C'est correct, comme nous l'avons vu plus tôt, le positionnement statique est le comportement par défaut !



- Le positionnement relatif est le premier type de positionnement que nous allons étudier. Il est très similaire au positionnement statique. Cependant, une fois que l'élément positionné occupe une place dans le cours normal de la mise en page, vous pourrez modifier sa position finale. Vous pourrez par exemple le faire chevaucher d'autres éléments de la page. Poursuivons : mettez à jour la déclaration de position dans `position: relative;`
- Si vous sauvegardez et actualisez à ce stade, vous ne verrez aucun changement dans le résultat. Alors, comment modifier la position de l'élément ? Vous avez besoin d'employer les propriétés `top`, `bottom`, `left` et `right`
- `top`, `bottom`, `left` et `right` sont utilisés conjointement à `position` pour définir exactement là où placer l'élément positionné. Pour le tester, ajoutez les déclarations suivantes à la règle `.positioned` dans le CSS :

```
position: relative;
```

```
top: 30px;
left: 30px;
```

## Positionnement relatif

Je suis élément de base de niveau bloc. Les éléments de niveau de bloc adjacents se trouvent sur de nouvelles lignes en dessous de moi.

Par défaut, je couvre 100% de la largeur de mon élément parent et je suis aussi haut que mon contenu enfant. Mes largeur et hauteur totales sont égales aux largeur et hauteur du contenu, plus celles du remplissage, plus celles de l'encadrement.

Entre éléments adjacents, nous sommes séparés par nos marges. En raison de la fusion des marges, nous sommes séparés par la largeur de la plus grande de nos marges, et non par la somme des deux.

Les éléments « inline » comme celui-ci ou cet autre sont sur une même ligne que les nœuds de texte adjacents, s'il y a de la place sur la même ligne. Les éléments « inline » débordants se replient, si possible sur une nouvelle ligne — comme celle-ci contenant du texte ; sinon, ils passent simplement à une nouvelle

ligne, un peu comme cette image le fait :





Le positionnement absolu nous apporte des résultats bien différents.

Appliquer position: absolute

```
position: absolute;
```

Modifions la déclaration de position dans le code :

## Positionnement absolu

Par défaut, je couvre 100% de la largeur de mon élément parent et je suis aussi haut que mon contenu enfant. Mes largeur et hauteur totales sont égales aux largeur et hauteur du contenu, plus celles du remplissage, plus celles de l'encadrement.

Je  
nouvelles lignes en dessous de moi.

Entre éléments adjacents, nous sommes séparés par nos marges. En raison de la fusion des marges, nous sommes séparés par la largeur de la plus grande de nos marges, et non par la somme des deux.

Les éléments « inline » comme celui-ci ou cet autre sont sur une même ligne que les noeuds de texte adjacents, s'il y a de la place sur la même ligne. Les éléments « inline » débordants se replient, si possible, sur une nouvelle ligne — comme celle-ci contenant du texte ; sinon, ils passent simplement à une nouvelle ligne, un peu comme cette image le fait :

